

Survey of Vector Database Management Systems

James Jie Pan · Jianguo Wang · Guoliang Li

Received: date / Accepted: date

Abstract There are now over 20 commercial vector database management systems (VDBMSs), all produced within the past five years. But embedding-based retrieval (EBR) has been studied for over ten years, and similarity search a staggering half century and more. Driving this shift from algorithms to systems are new data intensive applications, notably large language models (LLMs), that demand vast stores of unstructured data coupled with reliable, secure, fast, and scalable query processing capability. A variety of new data management techniques now exist for addressing these needs, however there is no comprehensive survey to thoroughly review these techniques and systems.

We start by identifying five main obstacles to vector data management, namely the vagueness of semantic similarity, large size of vectors, high cost of similarity comparison, lack of a natural partitioning that can be used for indexing, and difficulty of efficiently answering “hybrid” queries that require both attributes and vectors. Overcoming these obstacles has led to new approaches to query processing, storage and indexing, and query optimization and execution. For query processing, a variety of similarity scores and query types are now well understood; for storage and indexing, tech-

niques include vector compression, namely quantization, and partitioning techniques based on randomization, learning partitioning, and “navigable” partitioning; for query optimization and execution, we describe new operators for hybrid queries, as well as techniques for plan enumeration, plan selection, and hardware accelerated query execution. These techniques lead to a variety of VDBMSs across a spectrum of design and runtime characteristics, including “native” systems that are specialized for vectors and “extended” systems that incorporate vector capabilities into existing systems. We then discuss benchmarks, and finally we outline several research challenges and point the direction for future work.

1 Introduction

The rise of large language models (LLMs) [71] for tasks like information retrieval [36], along with the growth of unstructured data underlying economic drivers such as e-commerce and recommendation platforms [133, 125, 63], calls for new *vector database management systems* (VDBMSs) that can deliver traditional capabilities such as query optimization, transactions, scalability, fault tolerance, and privacy and security, but for unstructured data.

As these data are not represented by attributes from a fixed schema, they are not retrieved through structured queries but through similarity search, where data that have similar semantic meanings to the query are retrieved [95]. To support this type of search, entities such as images and documents are first encoded into D -dimensional feature vectors via an embedding model before being stored inside a VDBMS. The dual-encoder

James Jie Pan
Department of Computer Science and Technology
Tsinghua University, Beijing, China
E-mail: jamesjpan@tsinghua.edu.cn

Jianguo Wang
Department of Computer Science
Purdue University, West Lafayette, Indiana, USA
E-mail: csjgwang@purdue.edu

Guoliang Li
Department of Computer Science and Technology
Tsinghua University, Beijing, China
E-mail: liguoliang@tsinghua.edu.cn

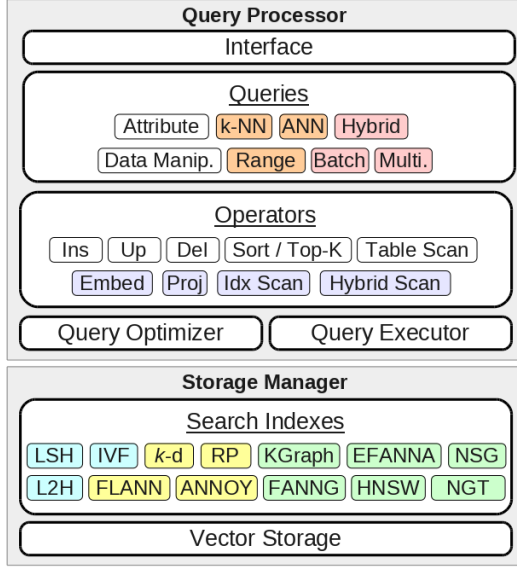


Fig. 1 Architecture of a VDBMS.

model [42] describes this procedure, also known as dense retrieval [73].

Consequently, the modules in a VDBMS split into a *query processor*, which includes the query specifications, logical operators, their physical implementations, and the query optimizer; and the *storage manager*, which maintains the search indexes and manages the physical storage of the vectors. This is illustrated in Figure 1.

The designs of these modules affect the runtime characteristics of the VDBMS. Many applications such as LLMs are read-heavy, requiring high query throughput and low latency. Others such as e-commerce are also write-heavy, requiring high write throughput. Additionally, some applications require high query accuracy, meaning that retrieved entities are true semantic matches to the query, while other applications may be more tolerant of errors. Developing a suitable VDBMS therefore requires understanding the landscape of techniques and how they affect characteristics of the system.

While there is mature understanding of processing conventional structured data, this is not the case for vector data. We present five key obstacles. (1) *Vague Search Criteria*. Structured queries use precise boolean predicates, but vector queries rely on a vague notion of semantic similarity that is hard to accurately capture. (2) *Expensive Comparisons*. Attribute predicates (e.g. $<$, $>$, $=$, and $\in$) can mostly be evaluated in $O(1)$ time, but a similarity comparison typically requires $O(D)$ time, where D is the vector dimensionality. (3) *Large Size*. A structured query usually only accesses a small number of attributes, making it possible to design read-efficient storage structures such as column stores. But vector search requires full feature vectors. Vectors some-

times even span multiple data pages, making disk retrievals more expensive while also straining memory. (4) *Lack of Structure*. Structured attributes are mainly sortable or ordinal, leading to partitionings via numerical ranges or categories that can be used to design search indexes. But vectors have no obvious sort order nor are they ordinal, making it hard to design indexes that are both accurate and efficient. (5) *Incompatibility with Attributes*. Structured queries over multiple attribute indexes can use simple set operations, such as union or intersection, to collect the intermediate results into the final result set. But vector indexes typically stop after finding k most similar vectors, and combining these with the results from an attribute index scan can lead to fewer than expected results. On the other hand, modifying the index scan operator to account for attribute predicates can degrade index performance. It remains unclear how to support “hybrid” queries over both attributes and vectors in a way that is both efficient and accurate.

There are now a variety of techniques that have been developed around these issues, aimed at achieving low query latency, high result quality, and high throughput while supporting large numbers of vectors. Some of these are results of decades of study on similarity search. Others, including hybrid query processing, indexes based on vector compression, techniques based on hardware acceleration, and distributed architectures are more recent inventions.

In this paper, we start by surveying these techniques from the perspective of a generic VDBMS, dividing them into those that apply to query processing and those that apply to storage and indexing. Query optimization and execution are treated separately from the core query processor. Following these discussions, we apply our understanding of these techniques to characterize existing VDBMSs.

Query Processing. The query processor mainly deals with how to specify the search criteria in the first place and how to execute search queries. For the former, a variety of similarity scores, query types, and query interfaces are available. For the latter, the basic operator is similarity projection, but as it can be inefficient, a variety of index-supported operators have been developed. We discuss the query processor in Section 2.

Storage and Indexing. The storage manager mainly deals with how to organize and store the vector collection to support efficient and accurate search. For most systems, this is achieved through vector search indexes. We classify indexes into *table-based* indexes such as E^2 LSH [49], SPANN [44], and IVFADC [69], that are generally easy to update; *tree-based* indexes such as FLANN [96], RPTree [47, 48], and ANNOY [1] that aim

to provide logarithmic search; and *graph-based* indexes such as KGraph [52], FANNG [66], and HNSW [90] that have been shown to perform empirically well but with less theoretical understanding.

To address the difficulty of partitioning vector collections, techniques include randomization [67, 49, 31, 96, 48, 52, 123, 115], learned partitioning [127, 69, 91, 96, 112], and what we refer to as “navigable” partitioning [51, 89, 90]. To deal with large storage size, several techniques have been developed for indexes over compressed vectors, including quantization [62, 69, 91, 113, 129, 133], as well as disk-resident indexes [61, 44]. We discuss indexing in Section 3.

Optimization and Execution. The query optimizer and executor mainly deals with plan enumeration, plan selection, and physical execution. To support hybrid queries, several hybrid operators have been developed, based on what we refer to as “block-first” scan [133, 125, 61] and “visit-first” scan [136]. There are also several techniques for enumeration and selection, including rule-based and cost-based selection [133, 125]. For query execution, several techniques aim to exploit the storage locality of large vectors to design hardware accelerated operators, taking advantage of capabilities such as processor caches [125], SIMD [125, 34, 35], and GPUs [70]. There are also distributed search techniques and techniques for supporting high throughput updates, namely based on out-of-place updates. We discuss optimization and execution in Section 4.

Current Systems. We classify existing VDBMSs into *native* systems which are designed specifically around vector management, including Vearch [81], Milvus [125], and Manu [63]; *extended* systems which add vector capabilities on top of an existing data management system, including AnalyticDB-V [133] and PASE [139]; and *search engines and libraries* which aim to provide search capability only, such as Apache Lucene [2], Elasticsearch [3], and Meta Faiss [4]. Native systems tend to favor high performance techniques targeted at specific capabilities, while extended systems tend to favor techniques that are more adaptable to different workloads but are not necessarily the fastest. We survey current systems in Section 5.

Related Surveys. A high level survey is available¹ that mostly focuses on fundamental VDBMS concepts and use cases. Likewise, some tutorials are available that focus specifically on similarity search [106, 107]. We complement these by focusing on specific problems and techniques related to vector data management as a whole. Surveys are also available covering data types that are related to vectors, such as time series and

Table 1 Common similarity scores.

Type	Score	Metric	Complexity	Range
Sim.	Inner Prod.	✗	$O(D)$	$\mathbb{R}$
	Cosine	✗	$O(D)$	$[-1, 1]$
Dist.	Minkowski	✓	$O(D)$	$\mathbb{R}^+$
	Mahalanobis	✓	$O(D^2 + O(1))$	$\mathbb{R}^+$
	Hamming	✓	$O(D)$	$\mathbb{N}$

strings, but not supported by VDBMSs. Unlike systems for these other data types, a VDBMS can make no assumptions² about feature vector dimensions. We refer readers to [54, 53].

For the remaining sections, we briefly discuss benchmarks in Section 6, followed by a summary of research challenges and open problems in Section 7. We conclude the survey in Section 8.

2 Query Processing

Query processing in a VDBMS begins with a search specification consisting of the *similarity score* and the *query type*. Search criteria are conveyed to the system through a *query interface*. Once a query is received by the system, it is processed by executing a chain of operators over the vector collection.

2.1 Similarity Scores

A similarity score $f : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ maps two D -dimensional vectors, $\mathbf{a}$ and $\mathbf{b}$, onto a scalar, $f(\mathbf{a}, \mathbf{b})$, with larger values indicating greater similarity.

Similarity is often measured via distance in practice, with values closer to 0 indicating greater similarity. Distance functions obey the metric axioms of identity ($d(\mathbf{a}, \mathbf{a}) = 0$), positivity ($d(\mathbf{a}, \mathbf{b}) > 0$ if $\mathbf{a} \neq \mathbf{b}$), symmetry ($d(\mathbf{a}, \mathbf{b}) = d(\mathbf{b}, \mathbf{a})$), and triangle inequality ($d(\mathbf{a}, \mathbf{c}) \leq d(\mathbf{a}, \mathbf{b}) + d(\mathbf{b}, \mathbf{c})$) for any three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$. Several similarity scores are commonly supported by VDBMSs, and these are summarized in Table 1.

Aside from these basic scores, some VDBMSs also support *aggregate scores* for applications like multi-vector search [129]. There is also emerging work on *learned scores* [25, 142, 93], but these are not available in commercial systems.

2.1.1 Basic Scores

The Hamming distance counts the number of differing dimensions between vectors $\mathbf{a}$ and $\mathbf{b}$.

¹ <http://arxiv.org/abs/2309.11322>

² *E.g.* the correlations in a time series.

Definition 1 (Hamming) $d(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n \delta_{a_i b_i}$

where δ is the Kronecker delta.

Other similarity scores start from an inner product. For $\mathbb{R}^D$ space, the dot product typically serves as the inner product.

Definition 2 (Inner Product) $f(\mathbf{a}, \mathbf{b}) = \sum_{i=1}^n a_i b_i$,

also written $f(\mathbf{a}, \mathbf{b}) = \langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a} \cdot \mathbf{b}$. The quantity $\sqrt{\langle \mathbf{a}, \mathbf{a} \rangle}$ defines the Euclidean norm $\|\mathbf{a}\|$ or magnitude of $\mathbf{a}$. Euclidean distance is the norm of the vector resulting from the subtraction of two vectors, $\|\mathbf{a} - \mathbf{b}\|$. Euclidean distance is metric.

The dot product projects $\mathbf{a}$ onto $\mathbf{b}$ and scales the result by the magnitude of $\mathbf{b}$. The scaling can lead to unintuitive consequences. For example, two large identical vectors have a larger dot product compared to two small identical vectors, thus they would be considered “more similar” under this definition.

If magnitude is not important, $\mathbf{a}$ and $\mathbf{b}$ can be normalized by $\hat{\mathbf{a}} = \mathbf{a}/\|\mathbf{a}\|$ and $\hat{\mathbf{b}} = \mathbf{b}/\|\mathbf{b}\|$ so that they possess unit magnitudes. Then,

Definition 3 (Cosine Similarity) $f(\mathbf{a}, \mathbf{b}) = \langle \hat{\mathbf{a}}, \hat{\mathbf{b}} \rangle$

or $f(\mathbf{a}, \mathbf{b}) = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \|\mathbf{b}\|}$. Cosine similarity yields the angle between $\mathbf{a}$ and $\mathbf{b}$.

Arbitrary p -norms, $\|\mathbf{x}\|_p = (|x_1|^p \dots |x_D|^p)^{1/p}$, induce the Minkowski distance, generalizing Euclidean distance.

Definition 4 (Minkowski) The p -order Minkowski distance is $d(\mathbf{a}, \mathbf{b}) = (\sum_{i=1}^n |a_i - b_i|^p)^{1/p}$,

or $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_p$. All positive and integer values of p yield metrics.

Another generalization of Euclidean distance can be obtained by applying a linear transformation over the vector space in order to adjust the relative proximities of the feature vectors. The distance of two vectors in the transformed space can be calculated using the Mahalanobis formula.

Definition 5 (Mahalanobis) For any positive semi-definite matrix $\mathbf{M}$, $d(\mathbf{a}, \mathbf{b}) = \sqrt{(\mathbf{a} - \mathbf{b})^\top \mathbf{M} (\mathbf{a} - \mathbf{b})}$.

2.1.2 Aggregate Scores

Sometimes, a single real-world entity is represented by multiple vectors in the vector collection. For example for facial recognition, a face may be represented by multiple images taken from different angles, leading to m feature vectors $\mathbf{x}_1 \dots \mathbf{x}_m$.

Given a query vector $\mathbf{q}$, finding the set of vectors which are collectively most similar to $\mathbf{q}$ is called *multi-vector search*³. One way of approaching this problem is to use an aggregate score that defines how to combine individual scores $f(\mathbf{x}_1, \mathbf{q}) \dots f(\mathbf{x}_m, \mathbf{q})$ to yield a single value that can be compared. Examples are the mean aggregate $1/m \sum_{i=1}^m f(\mathbf{x}_i, \mathbf{q})$ and the weighted sum [125].

2.1.3 Learned Scores

Some recent works [25, 142, 93] aim to find a transformation, $\mathbf{M}$, in order to improve the quality of similarity search over Mahalanobis distance. Finding $\mathbf{M}$ is one of the goals of *metric learning*, and we point interested readers to [121] for a survey of techniques.

2.1.4 Discussion

Score Selection. Many scores have been proposed over the years, but how to select an appropriate score for a particular application remains unclear. Ideally, a score is selected so that query answers accurately reflect the true semantic similarities between real-world entities represented by the vectors. But so far, there are no principles guiding score selection, and as a result, score selection tends to be based more on informal rules distilled from experience rather than on rigorous theory.

Many VDBMSs offer this as a choice to the user, and how to support automatic score selection remains an open problem. We are aware of one recent work [130] that dynamically adjusts the score based on the query for social media content recommendation.

At a higher level, vector search is affected not only by the similarity score but also by the nature of the embeddings and the semantics of the query. A preliminary discussion on query semantics is given in [116]. Hence, we imagine that future solutions will be more holistic, considering this problem from all aspects beyond score selection. For example, EuclidesDB [6] allows users to conduct the same search but over multiple embedding models and scores in order to identify the most semantically meaningful settings.

Curse of Dimensionality. When D grows beyond around 10 dimensions, and when the dimensions are independent and identically distributed, the Euclidean distances between the two farthest and two nearest vectors approach equality as the variance nears zero [41].

³ Some VDBMSs such as [5] offer a “hybrid” search that is a multi-vector search over dense feature vectors combined with sparse term vectors. The sparse vector is scored separately, *e.g.* by weighted term frequency, and then this score is combined with the feature vector similarity score to yield a final aggregate score.

The effect of this “curse of dimensionality” is that vectors become indiscernible⁴.

Attempts at avoiding the curse have led to using other Minkowski distances, such as the Manhattan distance ($p = 1$) or the Chebyshev distance ($p = \infty$), in an effort to recover discernability [27]. Fractional orders of $p < 1$ have also been explored [94] but so far, results are inconclusive.

2.2 Queries and Operators

Let $S \subset \mathbb{R}^D$ be a vector collection with N members. Let $\mathbf{q}$ be a D -dimensional query vector which may or may not belong to S .

Data manipulation queries aim to modify S . Given S , $\mathbf{q}$, and a way to measure similarity (whether by f or d), *vector search* queries aim to return a subset of S where each member of the subset satisfies criteria based on similarity to $\mathbf{q}$. Different criteria lead to different types of queries.

To answer these queries, a VDBMS may use several basic operators in addition to more sophisticated index operators that will be discussed in Section 3.

2.2.1 Data Manipulation Queries

Data manipulation queries provide insert, update, and delete mechanisms over S . In a traditional data management system, data is manipulated directly. But in a VDBMSs, feature vectors are proxies for the actual entities, and they can be manipulated either directly or indirectly. An *embedding model* maps real-world entities (*e.g.* images) to feature vectors.

Under direct manipulation, users freely manipulate the values of the vectors, and maintaining the model is the responsibility of the user. This is the case for systems such as PASE [139] and `pgvector` [7].

For indirect manipulation, vectors are hidden from users. The vector collection appears as a collection of entities, not vectors, and users manipulate the entities. The VDBMS is responsible for the model, which can be user-provided, for example through a user-defined function (UDF) as in Vald [8], or selected from a menu of pre-trained models. For example, Pinecone [9] supports a large number of pre-trained `*2vec` models⁵ by connecting to special providers⁶ via REST API.

There is extensive literature on designing embedding models, and we refer interested readers to [103].

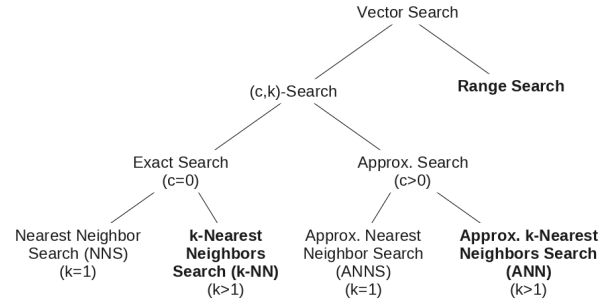


Fig. 2 Basic search queries.

2.2.2 Basic Search Queries

Several types of search queries exist, but not all VDBMSs support all types. The queries are shown in Figure 2.

Search queries can be viewed as either similarity maximization or distance minimization with respect to $\mathbf{q}$. We take the latter for the following definitions.

(c, k)-Search Queries. Most VDBMSs support “nearest neighbor” queries, where the aim is to retrieve vectors from S that are physical neighbors of $\mathbf{q}$ in the vector space. These queries may aim to return exact or approximate nearest neighbors, and may also specify the number of neighbors to return. We refer to these as (c, k) -search queries, where c indicates the approximation degree and k is the number of neighbors.

Out of these, most VDBMSs support the *approximate k-nearest neighbors* (ANN) query, which returns k vectors from S that are within a radius, centered over $\mathbf{q}$, of c times the distance between $\mathbf{q}$ and its closest neighbor.

Definition 6 (ANN) Find a k -size subset $S' \subseteq S$ such that $d(\mathbf{x}', \mathbf{q}) \leq c(\min_{\mathbf{x} \in S} d(\mathbf{x}, \mathbf{q}))$ for all $\mathbf{x}' \in S'$.

If $k = 1$, we call this query *approximate nearest neighbor search* (ANNS)⁷.

When $c = 1$, we call this an exact query. The case $c = 1, k = 1$ corresponds to nearest neighbor search (NNS) (see Note 7), and when $c = 1, k > 1$, the query is called a *k-nearest neighbors* (k -NN) query.

We note that there is a large literature on the *maximum inner product search* (MIPS) problem, which is the NNS query but over inner products. We refer interested readers to [117] for an overview.

Range Queries. A range query is parameterized by a radius, r , instead of the number of neighbors to return.

Definition 7 (Range) Find $\{\mathbf{x} \in S \mid d(\mathbf{x}, \mathbf{q}) \leq r\}$.

⁷ The terminology has become muddled over time. Early efforts focused on exact search, with $c = 1, k = 1$, which was referred to as “nearest neighbor search” (NNS) [67]. More recent efforts have focused on $c > 1, k > 1$, referring to this query as ANN, ANNS, or other names.

⁴ A diagram of this phenomenon is given in [97].

⁵ *E.g.* <https://github.com/MaxwellRebo/awesome-2vec>

⁶ *E.g.* <https://huggingface.co>.

Table 2 Query variants.

Variant	Sub-Variant
Predicated	
Batched	
Multi-Vector	Multi-Query, Single Feature (MQSF) Multi-Query, Multi-Feature (MQMF) Single-Query, Multi-Feature (SQMF)

2.2.3 Query Variants

Some VDBMSs support variations on these basic query types, listed in Table 2.

Predicated Search Queries. In a predicated search query, or “hybrid” query, each vector is associated with a set of attribute values, and a boolean predicate over these values must evaluate to true for each record in the result set⁸. Here is an example:

Example 1 A hybrid k -NN query written in SQL is:

```
select * from S where attr < c
order by d(q) limit k;
```

In this example, d is a distance function parameterized by the query q , and every member of the result set must satisfy the conditions of being among the k nearest and of obeying the predicate, $\text{attr} < c$.

Batched Queries. For batched queries, a number of queries are revealed to the system at the same time, and the VDBMS can answer them in any order.

These queries are especially suited to hardware accelerated query processing [125, 70].

Multi-Vector Queries. Some VDBMSs also support multi-vector search queries via aggregate scores.

There are three possible sub-types: in multi-query single-feature (MQSF) queries, the query is represented by multiple vectors, and real-world entities are represented by single feature vectors; in multi-query multi-feature (MQMF) queries, both the query and the entities are represented by multiple vectors; and in single-query multi-feature (SQMF) queries, only the entities are represented by multiple vectors. But so far, there is support for MQSF and SQMF queries [11, 5, 12, 125, 13] but no support for MQMF queries.

2.2.4 Basic Operators

It is possible to answer all (c, k) -search and range queries using only projection.

⁸ This type of query is called differently in different systems. In some VDBMSs such as [133], this is called a “hybrid” query, not to be confused with multi-vector search queries mentioned in Note 3. In others such as [10], this is called a “filtered” query.

Definition 8 (Projection) The projection of S onto g under query q yields $\pi_g(S) = \{g(\mathbf{x}, q) \mid \mathbf{x} \in S\}$.

Projection onto $g = f$ or $g = d$ alone is sufficient for vector search queries by projecting the full collection in time $O(\tau N)$, where τ is the cost of g .

Typically τ is in $O(D)$, making projection by itself impractical for VDBMSs where N and D are very large. Thus, most VDBMSs rely on specialized index-based operators in addition to projection.

2.2.5 Discussion

Query Accuracy and Performance. The search capability of a VDBMS is assessed by evaluating query accuracy and performance.

To evaluate accuracy, precision and recall are often used. Precision is defined as the ratio between the number of relevant results in the result set over the size of the result set, and recall is defined as the ratio between the number of retrieved relevant results over all possible relevant results. For example in a k -NN query, the precision is $k'/|S'|$, where k' is the number of true nearest neighbors in S' , and the recall is k'/k .

To evaluate performance, latency and throughput are used. Latency is the amount of time it takes for a VDBMS to answer a query once it is received, while throughput is the number of queries that are answered per unit time, often reported as queries per second.

Theoretical Results. Similarity search has a long history, making it possible to summarize some key theoretical results. For low-dimensional NNS, many theoretical results are known⁹. Algorithms with $O(\log N)$ query times and $O(N)$ storage are known for $D = 1$ (e.g. binary search trees) and $D = 2$ [84]. For the latter case, k -d trees [39] are particularly well known, and the query complexity is $O(\sqrt{N})$. For $D \geq 3$, sub-linear search performance is much harder to obtain. In the general case, k -d trees offer $O(DN^{1-1/D})$ query complexity [77], which tends toward $O(DN)$ as D grows. On the other hand, [92] offers $O(D^{O(1)} \log N)$ query complexity but requires super-polynomial $O(N^{O(D)})$ storage. The modern belief is that even a fractional power of N query complexity cannot be obtained *unless* storage cost is worse than $N^{O(1)} D^{O(1)}$ [32, 109].

Locality sensitive hashing (LSH) [67] is perhaps the most well understood ANN algorithm. The query complexity¹⁰ is dominated by N^ρ while the storage com-

⁹ Note that bounds on NNS also apply to k -NN.

¹⁰ Much effort has been on finding “families” of hash functions that can minimize ρ . For Hamming distance, [67] achieves $\rho \leq 1/c$, and for Euclidean distance, [29, 31] achieves $\rho \leq 1/c^2 + o(1)$. These are both optimal [100], although for Euclidean distance the

plexity is dominated by $N^{1+\rho}$, where $0 \leq \rho \leq 1$. Recently, heuristic methods which lack guarantees have become popular. There are encouraging analyses of graph-based techniques [75,104] but these lack the maturity of LSH.

2.3 Query Interfaces

For query interfaces, native and NoSQL VDBMSs tend to rely on small APIs. For example, Chroma [14] offers a Python API with just nine commands, including `add`, `update`, `delete`, and `query`.

On the other hand, extended VDBMSs built over relational systems tend to take advantage of SQL extensions. In `pgvector` [7], a k -NN or ANN query is expressed as:

```
select * from items order by
embedding <-> [3,1,2] limit 5;
```

The syntax `R <-> s` returns the Euclidean distance between all the tuples of `R` and vector `s`, and other distance functions are supported via other symbols. If an ANN index is created over the `items` table, then this query will return approximate results if the index is used for execution.

Similarly, range queries are expressed using `where`:

```
select * from items where
embedding <-> [3,1,2] < 5;
```

3 Indexing

While all (c,k) -search and range queries can be answered by comparing `q` with each of the N vectors, the complexity of this brute-force approach is $O(DN)$, prohibitive for large D and N .

Instead, vector indexes speed up queries by minimizing the number of comparisons. This is achieved by partitioning S so that only a small subset is compared, and then arranging the partitions into data structures that can be easily traversed.

Unlike typical attributes, vectors are not obviously sortable nor can they be easily categorized. To achieve high accuracy, these indexes rely on novel techniques

random projections family [49] is more popular due to its simplicity. This family achieves $\rho < 1/c$. These particular families are data-independent, but by exploiting structural relationships within a vector collection, [33] finds a family for Euclidean distance which yields $\rho = 1/(2c^2 - 1)$.

which we refer to as *randomization*, *learned partitioning*, and *navigable partitioning*. The large physical size of vectors also leads to use of compression, namely a technique called *quantization*, as well as *disk resident* designs. Additionally, the need to support predicated queries has led to special *hybrid operators* for indexes, which we will discuss in Section 4.

Partitioning Techniques.

- *Randomization*. Randomization aims to exploit probability amplification of multiple independent events, allowing indexes to better discriminate truly similar vectors from dissimilar ones.
- *Learned Partitioning*. Learning-based techniques aim to uncover an internal structure of S so that it can be partitioned along this structure. These techniques can be supervised or unsupervised.
- *Navigable Partitioning*. Instead of fixating on absolute partitions, navigable indexes are designed so that different regions of S can be easily traversed.

Some partitioning strategies are *data independent*, where the rules are the same for any data distribution. But the majority are *data dependent*. If updates to S alter its distribution, then indexes based on data dependent strategies may eventually become unbalanced over time. In many cases, this can only be resolved by rebuilding the index.

Storage Techniques.

- *Quantization*. A quantizer maps a vector onto a more space-efficient representation. Quantization is usually lossy, and the aim is to minimize information loss while simultaneously minimizing storage cost.
- *Disk-Resident Designs*. Compared to memory resident indexes which only minimize the number of comparisons, disk resident indexes additionally aim to minimize the number of retrievals.

Below, we examine the main techniques for some common indexes. One particular index may use a combination of techniques, and so we classify indexes based on their structure and then point out which techniques are used in which index. There are three basic structures: *tables* divide S into buckets containing similar vectors; *trees* are a nesting of tables; *graphs* connect similar vectors with virtual edges that can then be traversed. All of these structures are capable of achieving high query accuracy but with different construction, search, and maintenance characteristics.

3.1 Tables

The main consideration for table-based indexes is the design of the bucketing hash function. The most pop-

Table 3 Representative table-based indexes.

Type	Index	Hash Function
LSH	E ² LSH [49]	Rand. hyperplanes
	IndexLSH	Rand. bits
	FALCONN [31]	Rand. balls
L2H	SPANN [44]	Nearest centroid
Quant.	SQ	Nearest discrete value
	PQ	Nearest centroid product
	IVFSQ	Nearest centroid
	IVFADC [69]	Nearest centroid

ular table-based indexes for VDBMSs tend to use randomization and learned partitions, as shown in Table 3. For randomization, techniques based on LSH [30, 31, 32, 49, 67, 78, 88] are popular due to robust error bounds. For learned partitions, learning-to-hash (L2H) [127] directly learns the hash function, and indexes based on quantization [62, 69, 91, 113, 129, 133] typically uses k -means [40] to learn geometric clusters of similar vectors.

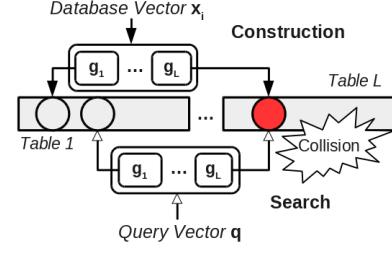
All of these indexes have similar construction and search characteristics. For construction, each vector is hashed into one of the buckets. The total complexity is typically $O(DN^{1+\epsilon})$, $0 \leq \epsilon \leq 1$. For LSH, each vector is hashed multiple times, leading to $\epsilon > 0$. For quantization-based approaches, k -means multiplies the complexity by a constant factor. For search, $\mathbf{q}$ is hashed onto a key and then the corresponding bucket is scanned. Hashing is generally on the order of $O(D)$. Usually only a small fraction of S is scanned, yielding a complexity of about $O(DN^\epsilon)$, including the cost of hashing and bucket scan.

3.1.1 Locality Sensitive Hashing

Locality sensitive hashing [67, 78] provides tunable performance with error guarantees, but it can require high redundancy in order to boost accuracy, increasing query and storage costs relative to other techniques.

In a “family” of hash functions $H = \{h : S \rightarrow U\}$, if $d(\mathbf{x}, \mathbf{q}) \leq r_1$, then $\Pr_H(h(\mathbf{x}) = h(\mathbf{q})) \geq p_1$, and if $d(\mathbf{x}, \mathbf{q}) \geq r_2$, then $\Pr_H(h(\mathbf{x}) = h(\mathbf{q})) \leq p_2$, for any $r_1, r_2, \mathbf{x} \in S$, and $\mathbf{q}$. A tunable family $G = \{g : S \rightarrow U^K\}$ is derived by letting $g(\mathbf{x})$ return the concatenation of $h_i(\mathbf{x})$ for i between 1 and some constant K .

The table is constructed by hashing each $\mathbf{x} \in S$ into each of the L hash tables using $g_1 \dots g_L$. Typically, L is set to $L = O(1/p_1^K)$ with K set to $\lceil \log_{1/p_2} N \rceil$ [32]. The exact value depends on the accuracy and performance needs of the application, and some sample curves are shown in [30]. Letting $\rho = \log(1/p_1)/\log(1/p_2)$ yields $L = O(N^\rho/p_1)$. The storage complexity is $O(LDN)$ which is $O(DN^{1+\rho})$ after substitution. In the practical case where $p_1 > p_2$, the value of ρ is between 0 and 1.

**Fig. 3** Constructing and searching an LSH index.

When a query appears, it is hashed using the L hash functions sampled from G , and collisions are kept as candidate neighbors. The candidates are then re-ranked or discarded based on true distances to $\mathbf{q}$. Figure 3 illustrates this procedure. The query complexity is dominated by the L hash evaluations, which is $O(DN^\rho)$.

When r_1 is set to $\min_{\mathbf{x} \in S} d(\mathbf{x}, \mathbf{q})$ and r_2 is set to cr_1 , the c guarantee is relative to the minimum distance. This is useful when the query is static across the workload, but is hard to generalize over dynamic on-line queries. Hence for an index designed around some given hash family, not all queries may have similar candidate sets, making it hard to control precision and recall. Multi-probe LSH [88] is one attempt at addressing this issue by scanning multiple buckets at a time, thereby spreading out the search.

We mention a few popular LSH schemes. The first two are *data independent* and require no rebalancing.

- *E²LSH*. Each g is an $O(D)$ projection onto a random hyperplane. This achieves $\rho < 1/c$ [49].
- *IndexLSH*. This scheme is based on binary projections and is provided by Faiss [4].

There have also been efforts at designing *data dependent* hash families to yield lower ρ .

- *FALCONN*. Implements an LSH hash family based on spherical LSH [31]. The dataset is first projected onto a unit ball and then recursively partitioned into small overlapping spheres. The ρ value is $1/(2c^2 - 1)$.

Other families are given in [32].

3.1.2 Learning to Hash

Learning-based techniques aim to directly learn suitable mappings without resorting to hash families. For example, spectral hashing [134] designs hash functions based on the principal components of the similarity matrix of S . In [110], hash functions are modeled using neural networks. In SPANN [44], vectors are hashed to the nearest centroid following k -means. Several techniques for a disk resident table-based index are also given.

These techniques tend to require lengthy training and are sensitive to out-of-distribution updates, and

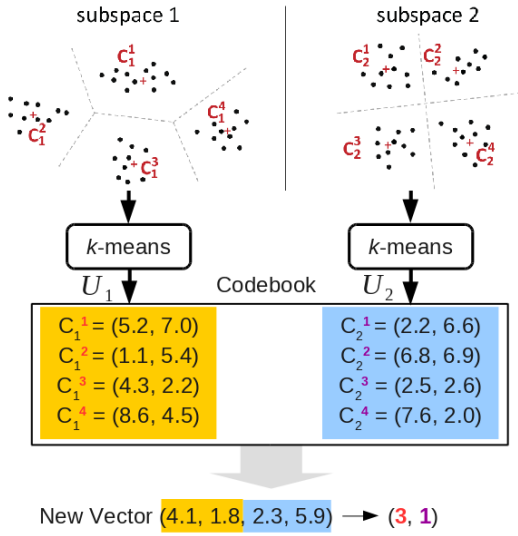


Fig. 4 In this example from [129], each 4-dimensional vector is divided into two 2-dimensional sub-spaces.

they are not widely supported in VDBMSs. We point readers to [127] for a survey of techniques.

3.1.3 Quantization

One of the main criticisms of LSH is that the storage cost can be large due to the use of multiple hash tables. For in-memory VDBMSs, large memory requirements can be impractical. While some efforts aim at reducing these storage costs [143], other efforts have targeted vector compression using quantization [62, 113, 69].

Many of these techniques use k -means centroids as hash keys¹¹. A large K number of centroids can modulate the chance of collisions, keeping buckets reasonably small and speeding up search. Normally, k -means is terminated once a locally optimal set of centroids is found, with complexity $O(DN \cdot K)$ per iteration.

But large K make k -means expensive. *Product quantization* exploits the fact that the cross product of m number of (D/m) -dimensional spaces is a space of D dimensions, so that by setting $U = \prod_{j=1}^m U_j$, then $U \in \mathbb{R}^D$ when $U_j \in \mathbb{R}^{D/m}$. This means that to yield a count of K centroids, only $K^{1/m}$ centroids need to be found per U_j . Moreover as each U_j belongs to a lower dimensional space, the running time of k -means per U_j is reduced. The new complexity is $O(m)O(\frac{D}{m}NK^{1/m})$.

Each U_j is constructed via k -means over the collection of sub-vectors $\{(x_i)_{i=(j-1)D/m+1}^{jD/m} \mid \mathbf{x} \in S\}$ ¹², and the set of all U_j is known as the “codebook”. Vector $\mathbf{x}$ is then quantized by splitting it into m sub-vectors,

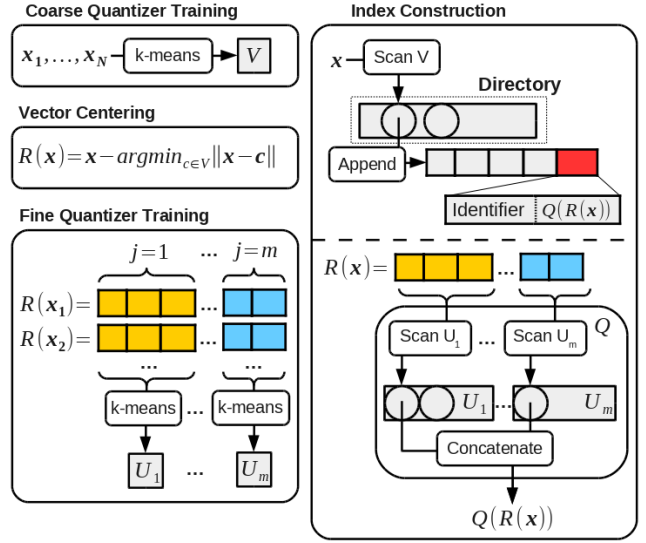


Fig. 5 Construction of an IVFADC index.

$\mathbf{x}'_j$, finding the nearest centroid in U_j to $\mathbf{x}'_j$ for each $j \in 1 \dots m$, and then concatenating these centroids. Each vector is thus stored using $m \log_2(D/m)$ bits, and the time complexity is $O(m)O(DK^{1/m})$. An example is shown in Figure 4.

Various techniques such as Cartesian k -means [98], optimized PQ (OPQ) [59], hierarchical quantizers [138], and anisotropic quantizers [64] have been developed based on this idea, offering around 60% better recall in the best cases at the cost of additional processing. A survey of these techniques is given in [91]. The storage cost can also be reduced by constant factors [129].

We list some quantization-based indexes.

- *Flat Indexes*. Faiss [4] supports a number of “flat” indexes where each vector is directly mapped onto its compressed version, without any bucketing. The standard quantizer index, SQ, performs a bit-level compression, for instance by mapping 64-bit doubles onto 32-bit floats¹³. The PQ index directly maps each vector onto its PQ code.
- *IVFSQ*. For IVFSQ, the vectors are compressed using SQ and bucketed to their nearest centroid.
- *IVFADC*. Training a PQ quantizer over S can still be time consuming. To reduce this cost, IVFADC first buckets vectors using k -means over a small number of centroids, and then trains a PQ quantizer by sampling a few vectors from each of the buckets. To allow a single quantizer to apply to all the buckets, each vector $\mathbf{x}$ is normalized by subtracting from its bucket key, resulting in a “residual” vector $R(\mathbf{x})$ which is then used to train the quantizer. The full workflow is shown in Figure 5. During search, query

¹¹ For a discussion, see [86, 62, 60].

¹² The notation $(x_i)_{i=1}^D$ expands to $x_1 x_2 \dots x_D$.

¹³ Also called “lattice” quantization, see [28].

$\mathbf{q}$ is directly compared against the quantized vectors in the bucket that $\mathbf{q}$ maps onto. As $\mathbf{q}$ itself is not quantized, the comparison is referred to as an “asymmetric distance computation” (ADC).

For IVFADC, many distance calculations are likely to be repeated during bucket scan since many vectors may share the same PQ centroids. These calculations can be avoided by first computing $\|\bar{\mathbf{q}}_j - \mathbf{c}\|^2$ for all $\mathbf{c} \in U_j$ and for all $j \in 1 \dots m$, where $\bar{\mathbf{q}}_j$ is the j th sub-vector of $\mathbf{q}$ [91]. This preprocessing step takes $O(m)O(\frac{D}{m}K')$, where K' is the number of centroids in U_j . But afterwards, ADC can be performed using just m look-ups, reducing bucket scan from $O(DN)$ to $O(mN)$.

Example 2 Below is the ADC look-up table when U is divided into m subsets, and where each subset contains K' centroids. Here, $\bar{\mathbf{q}}_j$ is the j th sub-vector of query $\mathbf{q}$, and $\mathbf{c}_i^j$ is the i th centroid in the j th subset of U .

$$\begin{array}{ccccccc} \overbrace{d(\bar{\mathbf{q}}_1, \mathbf{c}_1^1) \cdots d(\bar{\mathbf{q}}_1, \mathbf{c}_{K'}^1)}^{U_1} & & \overbrace{d(\bar{\mathbf{q}}_1, \mathbf{c}_1^m) \cdots d(\bar{\mathbf{q}}_1, \mathbf{c}_{K'}^m)}^{U_m} & & & & \\ \vdots & \ddots & \vdots & , \dots , & \vdots & \ddots & \vdots \\ d(\bar{\mathbf{q}}_m, \mathbf{c}_1^1) \cdots d(\bar{\mathbf{q}}_m, \mathbf{c}_{K'}^1) & & d(\bar{\mathbf{q}}_m, \mathbf{c}_1^m) \cdots d(\bar{\mathbf{q}}_m, \mathbf{c}_{K'}^m) & & & & \end{array}$$

We mention one other technique. In AnalyticDB-V [133], each bucket is further divided into finer sub-buckets in order to avoid accessing multiple full buckets. The resulting structure is called a “Voronoi Graph Product Quantization” (VG PQ) index.

3.2 Trees

For tree-based indexes, the main consideration is the design of the splitting strategy used to recursively split S into a search tree.

A natural approach is to split based on distance. The main techniques includes pivot-based trees [43, 144], such as VP-tree [140] and M-tree [45], k -means trees [96], and trees based on deep learning [80]. Other basic techniques are described in [111]. But while these trees are effective for low D , they suffer from the curse of dimensionality when applied to higher dimensions.

High- D tree-based indexes tend to rely on randomization for performing node splits. In particular, “Fast Library for ANN” (FLANN) [15, 96] combines randomization with learned partitioning via principal component analysis (PCA), extending the PKD-tree technique from [112], and “ANN Oh Yeah” (ANNOY) [1] is similar to the random projections tree (RPTree) from [47, 48]. These trees are summarized in Table 4.

The generic tree construction algorithm, from [47], is restated below:

Table 4 Representative tree-based indexes.

Index	Splitting Plane	Splitting Point
k -d tree [39]	Axis parallel	Median
PKD-tree [112]	Principal dim.	Median
FLANN [96]	Random principal dim.	Median
RPTree [47, 48]	Random plane	Median + offset
ANNOY [1]	Random plane	Random median

```

procedure MAKETREE( $S$ )
  if  $|S| \leq \tau$  then return Leaf
  end if
  Rule  $\leftarrow$  ChooseRule( $S$ )
  LeftTree  $\leftarrow$  MakeTree( $\{\mathbf{x} \in S \mid \text{Rule}(\mathbf{x}) \text{ is true}\}$ )
  RightTree  $\leftarrow$  MakeTree( $\{\mathbf{x} \in S \mid \text{Rule}(\mathbf{x}) \text{ is false}\}$ )
  return (Rule, LeftTree, RightTree)
end procedure

```

The complexity is characteristically $O(DN \log N)$, ignoring any preprocessing costs. More precise bounds for several trees are given in [108].

Most trees are able to return exact query results by performing backtracking, where neighboring leaf nodes are also checked during the search. However, this is inefficient [132], and they are more often used for returning approximate results using *defeatist search* [48]. In this procedure, the tree is traversed down to the leaf level, and all vectors within the leaf covering $\mathbf{q}$ are returned immediately as the nearest neighbors. There is no backtracking, and the complexity is $O(D \log N)$.

For maintenance, insertions require $O(D \log N)$ on average and $O(DN)$ in the worst case. But as node splits are determined during construction, there is no obvious way to rebalance the nodes after a number of out-of-distribution insertions. Self-balancing trees exist for scalar data [26] but not for high-dimensional vectors.

3.2.1 Non-Random Trees

Many high- D trees derive from k -d tree, which splits along medians while rotating through dimensions:

```

procedure CHOOSERULE( $S$ )
   $i \leftarrow l \bmod D$  where  $l$  is the current depth
  Rule  $:= (x_i \leq \text{median}(\{y_i \mid \mathbf{y} \in S\}))$ 
  return Rule
end procedure

```

This rule has the effect of fixing the splitting planes parallel to the dimensional axes [47, 108].

3.2.2 Random Trees

If certain dimensions explain the variance more than others, then the *intrinsic* dimensionality¹⁴ is lower than

¹⁴ A formal definition is given in [47].

D . But in this case, k -d tree is unable to partition along these dimensions, leaving it susceptible to the curse of dimensionality. This limitation has led to the discovery of more adaptive splitting strategies.

Principal Component Trees. A principal component tree is a k -d tree that is constructed by first rotating S so that the axes are aligned with the principal components of S . The principal dimensions need to be found beforehand using principal component analysis (PCA). The complexity of this step is $O(D^2N + D^3)$.

- *PKD-tree*. In PKD-tree, the splitting plane is selected by rotating through the principal dimensions, similar to in vanilla k -d tree [112].
- *FLANN*. Instead of rotating through the dimensions, FLANN [96] splits along random principal dimensions.

Random Projection Trees. On the other hand, random splitting planes can be used to adapt to the intrinsic dimensionality without expensive PCA.

- *RPTree*. RPTree [47,48] extends the idea of randomly rotated trees explored in [120] by introducing random splits in addition to random splitting planes. The principle follows from spill trees [85], where partitions are allowed to overlap. In RPTree, perturbed median splits simulate the effects of overlapping splits but with less storage cost. The splitting rule is [48]:

```

procedure CHOOSERULE( $S$ )
   $\mathbf{u} \leftarrow$  A vector from the unit sphere
   $\beta \leftarrow$  A number from  $[0.25, 0.75]$ 
   $v \leftarrow$  The  $\beta$  fractile of  $\pi_{\mathbf{u}}(S)$ 
  Rule  $:= (\mathbf{x} \cdot \mathbf{u} \leq v)$ 
  return Rule
end procedure

```

The values of $\mathbf{u}$, β , and $\mathbf{v}$ are uniformly chosen at random. The operation $\pi_{\mathbf{u}}(S)$ performs a projection of S onto $\mathbf{u}$. Variable v represents the perturbed median split, with $\beta = 0.5$ yielding the true median.

- *ANNOY*. Instead of splitting on the β fractile, ANNOY splits along the median of two random values sampled from $\pi_{\mathbf{u}}(S)$, which is simpler to compute.

Several theoretical results^{15,16} are known for RPTree, but it is not clear if these apply to ANNOY.

¹⁵ The rate at which RPTree fails to return a result set containing the true nearest neighbor, $\mathbf{x}'$, to query $\mathbf{q}$ is bounded by the “potential” of the query over S , defined as $1/(N-1) \sum_{i=2}^N \|\mathbf{q} - \mathbf{x}'\|^2 / \|\mathbf{q} - \mathbf{x}_i\|^2$. A detailed proof is available in [48].

¹⁶ During search, query $\mathbf{q}$ must be projected onto each unit vector at every level during traversal, leading to a search complexity of $O(D \log N)$. In [108], this is improved to $O(D \log D + \log N)$ by using a circular rotation which can be applied in $O(D \log D)$ time and achieves a similar effect as random projections. The trinary projection (TP) tree introduced in [124] similarly targets

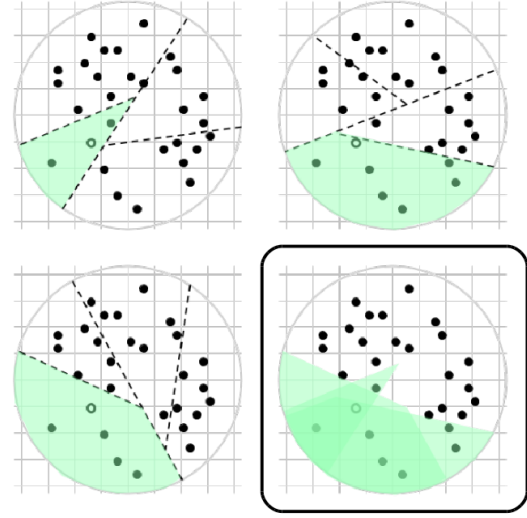


Fig. 6 As shown in [108], a forest of RPTrees can be used to improve recall by merging their individual result sets (bottom right) to the query (empty circle).

As shown in Figure 6, a forest of random trees can be used to improve recall. We mention that RPTree incurs a storage overhead of $O(DN)$ compared to k -d tree and FLANN due to storing the D -dimensional projection vectors, and this cost can be substantial for in-memory forests. In [72], several techniques are introduced to reduce this cost to $O(D \log N)$, for example by combining projections across the trees in a forest.

3.3 Graphs

A graph-based index is constructed by overlaying a graph on top of the vectors in $\mathbb{R}^D$ space, so that each node v_i is positioned over the vector $\mathbf{x}_i$ within the space. This induces distances over the nodes, $d(v_i, v_j) = d(\mathbf{x}_i, \mathbf{x}_j)$, which are then used to guide vector search along the edges. An example is shown in Figure 7.

The main consideration for these indexes is edge selection, in other words deciding which edges should be included during graph construction.

Graph-based indexes encapsulate all the partitioning techniques. Many graphs rely on random initialization or random sampling during construction. The k -nearest neighbor graph (KNNG) [56,101] associates each vector with its k nearest neighbors through an iterative refinement process similar to k -means and which we consider to be a form of unsupervised learning. Other

expensive $O(D)$ projections. Instead of projecting onto random or principal vectors, the splitting strategy partitions onto principal trinary vectors, which are vectors consisting of only -1 , 0 , or 1 . The principal trinary vectors can be approximated in $O(D)$ time. The search complexity remains $O(D \log N)$ but with smaller constant factors.

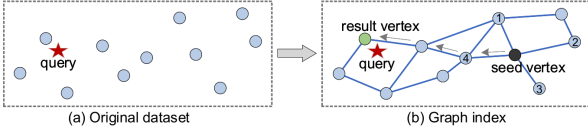


Fig. 7 In this example from [128], a graph is overlaid on top of the original dataset (a). Search begins from a “seed” vertex and is guided along the edges until it reaches the nearest node (vector) to the query (b).

Table 5 Representative graph-based indexes.

Type	Index	Initialization	Construction
KNNG	KGraph [52]	Random KNNG	Iterative refine
	EFANNA	Random trees	Iterative refine
MSN	FANNG [66]	Empty graph	Random trial
	NSG [58]	Approx. KNNG	Fixed trial
	Vamana [115]	Random graph	Fixed trial
SW	NSW [89]	Empty graph	One-shot refine
	HNSW [90]	Empty graph	One-shot refine

graphs, including monotonic search networks (MSNs) [51] and small-world (SW) graphs [89,90], aim to be highly navigable, but differ in their construction. The former tend to rely on search trials that probe the quality of the graph [66, 58, 115] while the latter use a heuristic procedure which we refer to as “one-shot refine”. Table 5 shows several graph indexes.

Graph-based search techniques have been shown to perform well in practice [82], and recent analytical results suggest that asymptotic performance approaches the N^ρ and $N^{1+\rho}$ limits achieved by LSH, albeit with $\rho = c^2/(2c^2 - 1)$ but smaller constant factors [75, 104]. An experimental comparison of graph-based techniques is available in [128].

3.3.1 k -Nearest Neighbor Graphs

In a KNNG, each node v_i is connected to k nodes representing the nearest neighbors to $\mathbf{x}_i$ [56]. For batched queries, $\mathbf{q}$ can be considered as a member of S , and a KNNG built over S allows exact k -NN search in $O(1)$ time through a simple look-up.

A KNNG can also be used to answer interactive queries, where $\mathbf{q} \notin S$. The basic idea is to recursively select node neighbors that are nearest to $\mathbf{q}$, starting from initial nodes, and add them into the top- k result set. The search complexity depends on the number of iterations before the result set converges. The search can start from multiple initial nodes, and if there are no more node neighbors to select, it can be restarted from new initial nodes [122].

A KNNG can be exact or approximated with a technique which we refer to as “iterative refine”.

Exact. An exact KNNG can be constructed by performing a brute force search N number of times, giving a total complexity of $O(DN^2)$. Unfortunately, there is little hope for improvement, as it is believed that the complexity is bounded by $N^{2-o(1)}$ [135]. An $O(N \log N)$ algorithm is given in [119] but with a constant factor that is $O(D^D)$. The algorithm in [102] achieves an empirical complexity of $O(N^{2-\epsilon})$, where $0 < \epsilon < 1$. This finding suggests the existence of efficient practical algorithms, despite the worst-case bounds.

Iterative Refine. An approximate KNNG can be obtained by iteratively refining an initial graph. We give two examples.

- *NN-Descent*. The NN-Descent (KGraph) method [52] begins with a random KNNG and iteratively refines it by examining the neighbors of the neighbors of each node v_i , replacing edges to v_i with edges to these second-order neighbors that are closer. When the dataset is growth restricted¹⁷, then each iteration is expected to halve the radius around each node and its farthest neighbor. This property leads to fast convergence, with empirical times on the order of $O(N^{2-\epsilon})$ for $0 < \epsilon < 1$.
- *EFANNA*. Instead of starting from a random KNNG, EFANNA¹⁸ uses a forest of randomized k -d trees to build the initial KNNG. Doing so is shown to lead to higher recall and faster construction, as it can quickly converge to better local optima. A similar approach is taken in [123] but where the tree is constructed via random hyperplanes.

3.3.2 Monotonic Search Networks

A KNNG is not guaranteed to be connected. Disconnected components complicates the search procedure for online queries by requiring restarts to achieve high accuracy [101, 122]. But by adding certain edges so that the graph is connected, it becomes possible to follow a single path beginning from any initial node and arriving at the nearest neighbor to $\mathbf{q}$.

A search path $v_1 \dots v_m$ is *monotonic* if $d(v_i, \mathbf{q}) > d(v_{i+1}, \mathbf{q})$ for all i from 1 to $m - 1$. An MSN is a graph where the search path discovered by a “best-first” search, in which the neighbor of v_i that is nearest to $\mathbf{q}$ is greedily selected, is always monotonic. This property implies a monotonic path for every pair of nodes in the graph and that the graph is connected.

The search complexity depends on the sum of the out-degrees over the nodes in the search path. If there

¹⁷ A growth restricted dataset is one where the number of neighbors of each node is bounded by a constant as the radius about the node expands.

¹⁸ <http://arxiv.org/abs/1609.07228>

are few total edges in the graph, then the search complexity is likely to be small. The minimum-edge MSN that guarantees exact NNS is believed to be the Delaunay triangulation [97]. But constructing a triangulation requires at least $\Omega(N^{\lceil D/2 \rceil})$ time [55], impractical for large N and D . As a result, several approximate methods have been developed, but these necessarily sacrifice the search guarantee.

In the early work by [51], an MSN is constructed in polynomial time by refining a sub-graph of the Delaunay triangulation called the relative neighborhood graph (RNG), introduced in [118] and later reviewed in [68]. The RNG itself is not monotone, but it can be constructed in $O(DN^{2-o(1)} \log^{1-o(1)} N)$ time under $\mathbb{R}^D$ Euclidean distance [114]. But the $N^{2-o(1)}$ term makes this approach impractical for large N .

Instead of repeatedly scanning the nodes, *search trials* can be used to probe the quality of the graph. Depending on the path taken by best-first search, new edges are added so that a monotonic path exists between source and target. The algorithm is shown below.

```

procedure MAKEMSN( $S$ )
   $G \leftarrow \text{InitializeGraph}(S)$ 
  repeat
     $(s, t) \leftarrow \text{ChooseSourceTargetPair}(S)$ 
     $P \leftarrow \text{GetSearchPath}(G, (s, t))$ 
     $\text{UpdateOutNeighbors}(G, t, P)$ 
    for each  $p \in P$  do
       $\text{UpdateOutNeighbors}(G, p, t)$ 
    end for
  until  $\text{Terminate}()$ 
  return  $G$ 
end procedure

```

For `InitializeGraph`, some indexes begin with an empty graph [66], random graph [115], or approximate KNNG [58]. Simple graphs can be initialized quickly but more complex graphs may offer better quality. For `ChooseSourceTargetPair`, one way is to select random pairs [66], while another is to designate a node as the source for all search trials [115, 58]. We refer to these techniques as *random* and *fixed* trials, respectively.

Random Trial. Indexes based on random trials are constructed over a large number of iterations, with each one leading to closer approximations of an MSN. The construction time can thus be adjusted with respect to the quality of the graph.

- *FANNG*. In the Fast ANN Graph [66], graph construction terminates after a fixed number of trials, e.g. $50N$. The `UpdateOutNeighbors` routine adds an edge between t and the nearest node in the search path, $p^* \in P$, and then prunes out-neighbors of p^* based on “occlusion” rules derived from the triangle

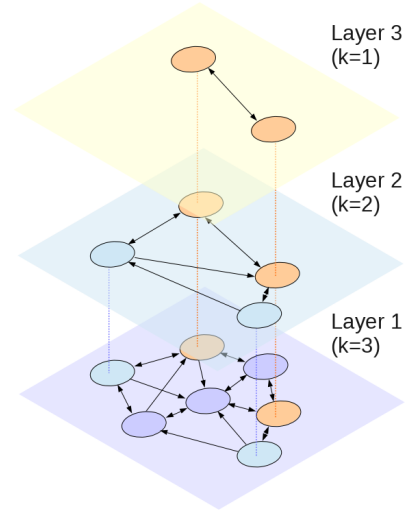


Fig. 8 Example HNSW index. Two nodes are randomly assigned to the top layer (orange), two to the middle layer (blue), and three to the bottom layer (purple). The out-degrees in each layer are bounded by k to modulate search complexity. Vertical edges allow the search algorithm to traverse down the layers, while horizontal edges allow traversal within a layer.

inequality in order to limit out-degrees. The empirical storage and search complexities are reported to be on the order of $O(DN^{1-\epsilon})$.

Fixed Trial. In fixed trial construction, all trials are conducted from a special designated source node, sometimes called the “navigating” node. This node also serves as the source for all online queries. Normally, the index is constructed by conducting one trial to each node in a single pass over S . The construction complexity is generally about $O(DN^{1+\epsilon} \log N^\epsilon)$ where the logarithmic term represents the cost of the search trials.

- *Navigating Spreading-Out Graph*. The NSG index [58] starts from an approximate KNNG. For `UpdateOutNeighbors`, it uses an edge selection strategy based on lune membership, similar to [51]. To guarantee that all targets are reachable from the navigating node, it overlays a spanning tree to connect any unreachable targets.
- *Vamana*. To speed up construction, Vamana [115] begins with a random graph instead of an approximate KNNG, and instead of checking lune membership, `UpdateOutNeighbors` uses a simple distance-based threshold similar to FANNG [66].

3.3.3 Small World Graphs

A graph is *small-world* if the length of its characteristic path grows in $O(\log N)$ [131]. A *navigable* graph is one where the length of the search path found by the best-first search algorithm scales logarithmically with N [74].

Table 6 Representative search indexes.

Structure	Index ^a	Partitioning	Residence	Constr.	Complexity ^b		Update ^c	Error Bound
					Space	Query		
Table	E ² LSH [49]	Space	Mem.	Med.	High	Med.	Y	✓
	FALCONN [33]	Space	Mem.	Med.	High	Med.	R	✓
	*SQ	Discrete	Mem.	Med.	Low	Med.	Y	✗
	*PQ	Clustering	Mem.	Med.	Low	Med.	R	✗
	*IVFSQ	Clustering	Mem.	Med.	Low	Med.	R	✗
	*IVFADC [69]	Clustering	Mem.	Med.	Low	Med.	R	✗
Tree	SPANN [44]	Clustering	Disk	Med.	Med.	Med.	R	✗
	FLANN [96]	Space	Mem.	High	High	Low	R	✗
	RPTree [47, 48]	Space	Mem.	Low	High	Low	R	✓
	*ANNOY	Space	Mem.	Low	High	Low	R	✗
Graph	NN-Descent (KGraph) [52]	Proximity	Mem.	Med.	Med.	Med.	N	✗
	EFANNA	Proximity	Mem.	High	Med.	Low	N	✗
	FANNG [66]	Proximity	Mem.	High	Med.	Med.	N	✗
	NSG [58]	Proximity	Mem.	High	Med.	Low	N	✗
	Vamana (DiskANN) [115]	Proximity	Disk	Med.	Med.	Low	N	✗
	*HNSW [90]	Proximity	Mem.	Low	Med.	Low	Y	✗

^a An asterisk (*) indicates supported by more than two commercial VDBMSs.

^b Based on theoretical results reported by authors, empirical results reported by authors, or our own cursory analysis when no results are reported. Key for complexity columns, with $0 \leq \epsilon \leq 1$ and a natural constant, K :

– *Construction*. High=worse than $O(DN^{1+\epsilon})$, Med.= $O(DN^{1+\epsilon})$, Low= $O(DN \log N)$;

– *Size*. High= $O(DN \cdot K)$ or worse, Med.= $O((D + K) \cdot N)$, Low= $O(N \log D)$;

– *Query*. High=worse than $O(DN^\epsilon)$, Med.= $O(DN^\epsilon)$, Low= $O(D \log N)$.

^c Y=data independent updates; R=updates with rebalancing; N=no updates.

A graph that is both navigable and small-world (NSW) thus possesses a search complexity that is likely to be logarithmic, even in the worst case.

- *NSW*. An NSW graph can be constructed using a procedure which we call *one-shot refine* and detailed in [89]. Nodes are sequentially inserted into the graph, and when a node is inserted, it is connected to its k nearest neighbors already in the graph.
- *HNSW*. While NSW offers search paths that scale in $\log N$, the out-degrees also tend to scale in the logarithm of N , leading to polylogarithmic search complexity. In [90], a hierarchical NSW (HNSW) graph is given which uses randomization in order to restore logarithmic search. During node insertion, the node is assigned to all layers below a randomly selected maximum layer, chosen from an exponentially decaying distribution so that the size of each layer grows logarithmically from top to bottom. Within each layer, the node is connected to its neighbors following the NSW procedure, but where the out-degrees are bounded. Best-first search proceeds from the top-most layer. Figure 8 shows an example.

3.4 Discussion

As seen from Table 6, HNSW offers many appealing characteristics. It is easy to construct, has reasonable storage requirements, can be updated, and supports

fast queries. It therefore comes as no surprise that it is supported by many commercial VDBMSs. The storage cost may still be a concern for very large vector collections, but there are ways to address this¹⁹.

Even so, there are cases where other indexes may be more appropriate. For batched queries or workloads where the queries belong to S , KNNs may be preferred, as once they are constructed, they can answer these queries in $O(1)$ time. KGraph is easy to construct, but EFANNA is more adaptable to any online queries. For online workloads, the choice rests on several factors. If error guarantees are important, then an LSH-based index or RPTree can be considered. If memory is limited, then a disk-based index such as SPANN or DiskANN may be appropriate. If the workload is write-heavy, then table-based indexes may be preferred, as they generally can be efficiently updated. Out of these, E²LSH is data independent and requires no rebalancing. For read-heavy workloads, tree or graph indexes may be preferred, as they generally offer logarithmic search complexity.

Aside from these indexes, there have also been efforts at mixing structures in order to achieve better search performance. For example, the index in [38] and the Navigating Graph and Tree (NGT) index [16] use a tree to initially partition the vectors and then use a graph index over each of the leaf nodes.

¹⁹ For example, Weaviate [5] allows constructing HNSW graphs over vectors that have been compressed with PQ.

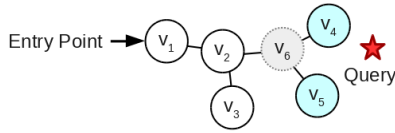


Fig. 9 Under best-first search, blocking out node v_6 makes the blue cluster of nodes, comprising the true nearest neighbor, unreachable from the entry point.

4 Query Optimization and Execution

There may be multiple ways to execute a given query. The goal of the query optimizer is to select the optimal query plan, typically the latency minimizing plan.

To achieve this goal, the first step is *plan enumeration*, followed by *plan selection* and then *query execution*. For predicated queries, vector indexes can not be easily combined with attribute filters in the same plan, resulting in the development of new *hybrid* operators.

4.1 Hybrid Operators

Predicated queries can be executed by either applying the predicate filter before vector search, known as “pre-filtering”; after the search, known as “post-filtering”; or during search, known as “single-stage filtering”.

If the search is index-supported, then there needs to be a mechanism to inform the index that certain vectors are filtered out. For pre-filtering, *block-first scan* works by “blocking” out vectors in the index before the scan is conducted [133, 125, 61]. The scan itself proceeds as normal but over the non-blocked vectors. For single-stage filtering, *visit-first scan* works by scanning the index as normal, but meanwhile checking each visited vector against the predicate conditions [136].

4.1.1 Block-First Scan

Blocking can be done online at the time of a query, or if predicates are known beforehand, it can be done offline.

Online Blocking. For online blocking, the aim is to perform the blocking as efficiently as possible in order to minimize the impact on query latency. In AnalyticDB-V [133] and Milvus [12, 125], a technique using bitmasks is given. A bitmask is constructed using traditional attribute filtering techniques. Then, during index scan, a vector is quickly checked against the bitmask to determine whether it is “blocked”.

Offline Blocking. For graph-based indexes, blocking can cause the graph to become disconnected, as shown in Figure 9. In Filtered-DiskANN [61], the aim is to prevent disconnections in the first place by strategically

adding edges based on the attribute category of adjoining nodes. A similar preventative approach is used in Qdrant [10] for HNSW and in [136].

In Milvus [12, 125], S is pre-partitioned along attributes that are expected to be predicate targets. When a query arrives, it can then be executed on the relevant partition using a normal index scan.

4.1.2 Visit-First Scan

For low-selectivity predicates, visit-first scan can be faster than online blocking because there is no need to block the vectors beforehand. But if the predicate is highly selective, then visit-first scan risks frequent backtracking as the scan struggles to fill the result set.

One way to avoid backtracking is to infuse the scan operator with a traversal mechanism that incorporates attribute information. In [61], the filter condition is added to the best-first search operator. In [136], the distance function used for edge traversal is augmented with an attribute related component so that the scan favors nodes that are likely to pass the filter²⁰.

4.2 Plan Enumeration

As vector search queries tend to consist of a small number of operators, in many cases *predefining* the plans is not only feasible but also efficient, as it saves overhead of enumerating the plans online. But for systems that aim to support more complex queries, the plans cannot be predetermined. For extended VDBMSs based on relational systems, relational algebra can be used to express these queries, allowing *automatic* enumeration.

4.2.1 Predefined

For predefined plans, the main consideration is which plan to specify for which query. Some systems target specific workloads, thereby focusing on single plans per query. Other systems predefine multiple plans.

Single Plan. Single plans can be highly efficient as it removes the overhead of plan selection in addition to enumeration, but can be a disadvantage if the predefined plan is not suited to the particular workload.

A non-predicated query trivially has a single query plan when only one type of search method is available. For example in EuclidesDB [6], each database instance is configured with one search index which is used for every search query. This can also be true for predicated queries. For example in Weaviate, all predicated search queries are executed by pre-filtering. Meanwhile

²⁰ See also <http://arxiv.org/abs/2203.13601>.

in Vearch [17,81], all predicated search queries are executed using post-filtering.

Multiple Plans. For non-predicated queries, different indexes lead to multiple plans. For example, AnalyticDB-V [133] supports brute force scan and table-based index scan over PQ or VGPQ. This allows a k -NN query to be executed using either of these methods.

Predicated queries can be answered either by pre-filtering, post-filtering, or single-stage filtering. But different vector search indexes, along with the presence or absence of an attribute index, multiply the number of possible plans.

4.2.2 Automatic

For automatic enumeration, some VDBMSs based on relational systems take advantage of the underlying relational optimizer to perform plan enumeration as well as selection. The relational language is extended to support distance functions and vector index scans. For example, `pgvector` [7] and PASE [139] both take advantage of PostgreSQL support for user extensions.

4.3 Plan Selection

To identify the optimal query plan, existing VDBMSs perform plan selection either by using *handcrafted rules* or by using a *cost model*.

4.3.1 Rule Based

If the number of plans is small, then selection rules can be used to decide which plan to execute. Figure 10 shows two examples, used by Qdrant [10] (Figure 10a) and Yahoo Vespa [18] (Figure 10b).

Both Qdrant and Vespa perform plan selection based on the estimated selectivity of the predicate. In Qdrant, a predicated k -NN query can be executed by a brute force scan, pre-filtering with an attribute index, or pre-filtering with HNSW. Plan selection is based on two thresholds, one on the size of S and the other on the selectivity of the filter. If S is small, then a brute force scan is performed. On the other hand, if the selectivity is high, meaning a small fraction of S passes the filter, then the attribute index is used, otherwise HNSW is used. In Vespa, a predicated query can additionally be executed by using HNSW followed by post-filtering when the selectivity estimate is low. Moreover, if the estimate does not immediately favor brute force scan or post-filtering, then Vespa will apply the filter, and then choose between a brute force scan over the filtered collection or a blocked HNSW scan based on the actual

selectivity. Existing methods [46] are used for estimating selectivity.

4.3.2 Cost Based

Plan selection can also be performed using a cost model, choosing the plan with the least estimated cost.

In AnalyticDB-V [133] and Milvus [12,125], a linear cost model sums the component costs of the individual operators to yield the cost of each plan. The basic operator cost depends on the number of distance calculations as well as memory and disk retrievals performed by the operator. For predicated queries, these numbers are estimated from the selectivity of the predicate. But they also depend on the desired query accuracy, which is exposed to the user as an adjustable parameter. The effect of different accuracy levels on operator cost is determined offline.

We point out several unaddressed challenges to cost estimation for predicated queries. For pre-filtering with best-first search, the cost of the scan can be hard to estimate due to uncertainty around the amount of blocking. This applies more to tree or graph-based indexes, as for table-based indexes, the cost of a scan is bounded above by bucket size. Likewise for visit-first scan, the cost of the scan depends on the rate of predicate failures, which is hard to know beforehand.

On the other hand, post-filtering for a predicated k -NN query may lead to a result set that contains fewer than k items. In VDBMSs that use post-filtering, this is often mitigated by retrieving αk nearest vectors instead of just the k nearest. But higher α make search more expensive, and there is no clear way for deciding on the optimal value which minimizes search cost while guaranteeing k results in the final result set.

4.4 Query Execution

Vector operators can take advantage of *hardware acceleration* such as via processor caches, SIMD instructions, and GPUs in order to reduce query latency. A VDBMS can also use *distributed search* to reduce the computing burden for a single machine, and to increase throughput, multiple searches can be conducted in parallel over the distributed cluster. For write-heavy workloads, a VDBMS can sacrifice consistency for write throughput by using *out-of-place updates*, which allow index updates to be deferred until more suitable times.

4.4.1 Hardware Acceleration

Vector comparison requires reading full vectors into the processor. This aspect of vectors along with large size

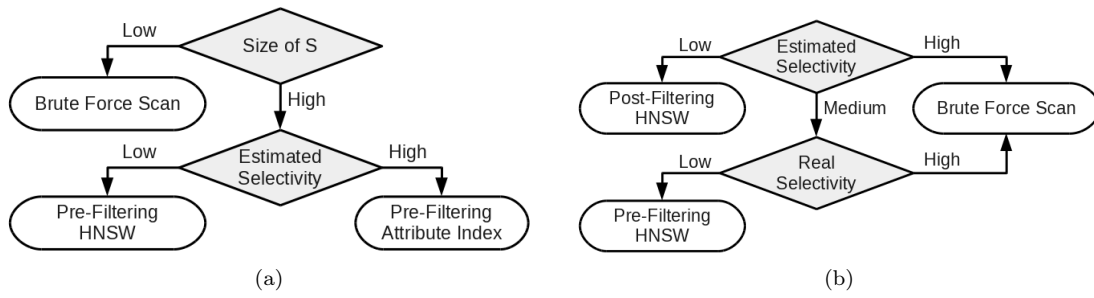


Fig. 10 Plan selection rules in (a) Qdrant and (b) Vespa.

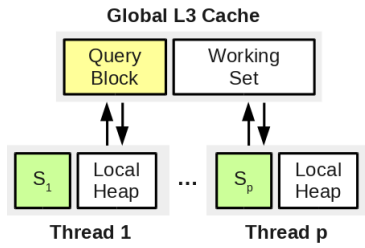


Fig. 11 Cache friendly query blocks used by Milvus [12, 125].

complicates disk retrieval, but this same locality makes them amenable to hardware accelerated processing.

CPU Cache. If data is not present in the processor cache, then it must be retrieved from memory, stalling the processor. As shown in Figure 11, Milvus [12, 125] minimizes cache misses for batched queries by partitioning the queries into query blocks, which are small enough to fit into the CPU cache. The queries are answered a block at a time, and multiple threads can be used to process the queries. As each thread references the entire block when performing a search, the block is safe from eviction under the common eviction policies.

Single Instruction Multiple Data (SIMD). The original ADC algorithm performs a series of table look-ups and summations (see Example 2). While SIMD instructions can trivially parallelize the summations, look-ups require memory retrievals (in the case of cache misses) and are more difficult to speed up. But in [34, 35], the SIMD shuffle instruction is cleverly exploited to parallelize these look-ups within a single SIMD processor. This technique is implemented in Faiss [4].

The basic idea is illustrated in Figure 12. The look-up indices, plus the entire look-up table, are stored into the SIMD registers. The shuffle operator is then used to rearrange the values of the table register so that the i th entry contains the value at the i th index, lining up the values for the subsequent additions.

The table is aggressively compressed in order to fit it into a register. First, centroids are represented using 4 bits instead of a more customary 8 bits, yielding a

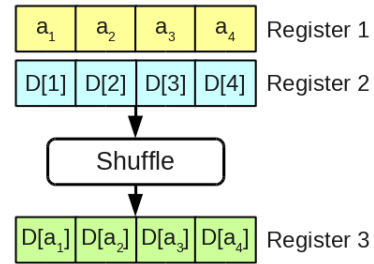


Fig. 12 Performing table look-up with SIMD.

total of $2^4 = 16$ centroids. Second, the distances in the table are quantized into 8-bit integers. The result is that there are only 16 8-bit integers in the look-up table, fitting inside a typical 128-bit register. In [35], some improvements are made to allow more values to be stored in the register, namely variable-bit centroids and splitting up large tables across multiple registers.

Graphical Processing Units (GPUs). A GPU consists of a large number of processing units in addition to a large device memory. The threads within a processing unit are grouped together in “warps”, and each warp has access to a number of 32-bit registers shared across the threads. An architectural diagram can be found in [83].

In [70], an ADC search algorithm for GPUs is given, also as a part of Faiss. Similar to the SIMD algorithm, the GPU algorithm likewise tries to avoid memory retrievals, this time from GPU device memory. It also achieves this by performing table look-ups within the registers, taking advantage of a shuffle operator called “warp shuffle”.

If the running k nearest neighbors are tracked in the registers, then k cannot be too large. Milvus [12, 125] works around this issue by conducting a k -NN over multiple rounds, flushing intermediate results store in the registers into host memory after each round.

4.4.2 Distributed Search

Many VDBMSs [8, 9, 12, 125, 63, 10, 18, 133] take advantage of distributed clusters in order to scale to larger

datasets or greater workloads. Some that are offered as a cloud service take advantage of disaggregated architectures in order to offer high elasticity. We point readers to [126] for details about these architectures.

To perform a distributed search, the vector collection is first partitioned into shards. The collection can be partitioned into equal shards, where the vectors in the shards are identically distributed, or partitioned based on other characteristics, for example based on index key for collections that are bucketed using a table-based index. A local index can then be built for each shard, and shards and their local indexes can also be replicated to provide fault tolerance and to increase throughput, as multiple queries can be executed simultaneously over the replicas.

Distributed vector search follows a scatter-gather pattern. The query is first scattered to all the relevant shards, then the result set is obtained by aggregating the results from each shard. For example for a k -NN query, each shard produces a result set containing the k nearest neighbors to the query that are members of the shard, and then a central coordinator gathers and merges these results to produce the final result set.

4.4.3 Out-Of-Place Updates

Updating an index in-place, even for fast update structures such as HNSW, can disrupt search queries if the index cannot be used during this time. These disruptions can be severe if updates take a long time or during a lengthy rebuild.

Replicas. Some VDBMSs mitigate this issue by partitioning the vector collection into shards and replicas, and then constructing a local index over each replica [8,5,133]. In this way, if the index on one replica is undergoing an update or rebuild, queries can be handled by a different replica without any disruption. But the storage (memory) requirements are multiplied by the number of replicas, and there may be extra overhead for search queries due to scatter-gather.

Log-Structured Merge (LSM) Tree. Another approach is to stream updates into a separate structure and then reconciled against the index at a more convenient time. An LSM tree [99,87] solves the problem that read-friendly indexes cannot support fast writes, and write-friendly update structures such as differential files and append-only logs cannot support fast reads.

In Milvus [12,125] and Manu [63], a vector search index is constructed over each segment of the LSM tree in order to support fast reads. A new index is created whenever a segment becomes full or is merged.

Other Techniques. In Vald [8], updates are streamed into a simple queue, and then applied in bulk to the lo-

cal index once the queue is full or based on other conditions. Likewise in AnalyticDB-V [133], updates are kept in memory and periodically merged with older records that are kept on disk.

5 Current Systems

The variety of data management techniques has led to an equally diverse landscape of commercial VDBMSs. They can be broadly categorized as *native* systems, which are designed specifically for vector data management, and *extended* systems, which add vector capabilities on top of an existing system. Table 7 lists several of these systems.

5.1 Native

Native systems are characterized by small query APIs, simple processing flows composed of a low number of components, and basic storage models, making them highly specialized at vector data management.

These systems can be further divided into two sub-categories, those that target *mostly vector* workloads [6,17,9,8,14], where the vast majority of queries access the vector collection, and those that target *mostly mixed* workloads [5,12,13,10,63,11], where queries are also expected to access non-vector collections. Mostly mixed workloads may consist of traditional attribute queries or textual keyword queries, along with typical predicated and non-predicated vector queries.

5.1.1 Mostly Vector

Mostly-vector systems are designed to support fast search queries over large vector collections. Several systems, such as EuclidesDB [6] and Vald [8], focus exclusively on non-predicated queries. Others, such as Vearch [17], Pinecone [9], and Chroma [14] offer predicated queries but supported by a single predefined plan. These systems often support only a single search index, typically graph-based. Hence, they have no need for a query parser, rewriter, or optimizer, and can omit these components in order to reduce processing overhead. They also typically do not support exact search, as all queries are handled by the index.

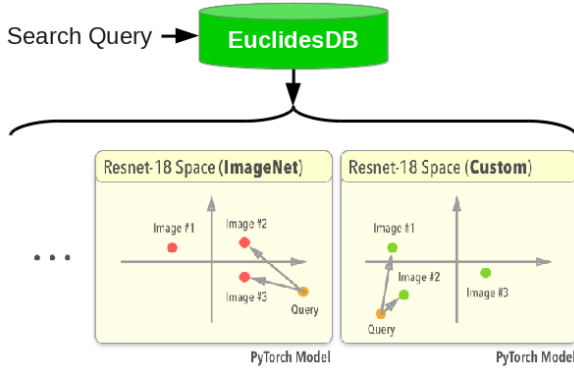
We point out some examples.

- *EuclidesDB*. EuclidesDB [6], shown in Figure 13, is a system for managing embedding models, aimed at allowing users to experiment over various models and similarity scores in order to find the most semantically meaningful setting. Users bring their own

Table 7 A list of current VDBMSs. Dates are estimated from paper publication, earliest Github release, blog post, or other indications. Dates indicate year when vector search capability first appeared in the product.

Name	Type	Sub-Type	Vector Query			Query Variant			Vector Index			Opt.
			Ex.	Ap.	Rng.	Pr.	Mul.	Bat.	Tab.	Tr.	Gr.	
EuclidesDB (2018) [6]	Nat.	Vec.	✓	✓	✗	✗	✗	✗	✓	✓	✓	✗
Vearch (2018) [17,81]	Nat.	Vec.	✗	✓	✗	✓	✗	✗	✓	✗	✗	✗
Pinecone (2019) [9]	Nat.	Vec.	✗	✓	✗	✓	✓	✗	Proprietary			U
Vald (2020) [8]	Nat.	Vec.	✓	✓	✓	✗	✗	✓	✗	✗	✓	✗
Chroma (2022) [14]	Nat.	Vec.	✗	✓	✗	✓	✗	✗	✗	✗	✓	U
Weaviate (2019) [5]	Nat.	Mix	✗	✓	✓	✓	✓	✗	✗	✗	✓	✗
Milvus (2021) [12,125]	Nat.	Mix	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
NucliaDB (2021) [13]	Nat.	Mix	✗	✓	✓	✓	U	✗	✗	✗	✓	✗
Qdrant (2021) [10]	Nat.	Mix	✓	✓	✓	✓	✓	✓	✗	✗	✓	✓
Manu (2022) [63]	Nat.	Mix	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
Marqo (2022) [11]	Nat.	Mix	✗	✓	✗	✓	✓	✓	✗	✗	✓	U
Vespa (2020) [18]	Ext.	NoSQL	✓	✓	✓	✓	✓	✗	✗	✗	✓	✓
Cosmos DB (2023)	Ext.	NoSQL	✗	✓	✗	✗	✗	✗	✓	✗	✗	✗
MongoDB (2023)	Ext.	NoSQL	✗	✓	✗	✓	✗	✗	✗	✗	✓	✗
Neo4j (2023)	Ext.	NoSQL	✗	✓	✗	✗	✗	✗	✗	✗	✓	✗
Redis (2023)	Ext.	NoSQL	✓	✓	✓	✓	✗	✓	✗	✗	✓	✗
AnalyticDB-V (2020) [133]	Ext.	Rel.	✓	✓	✓	✓	✗	✗	✓	✗	✓	✓
PASE+PG (2020) [139]	Ext.	Rel.	✓	✓	✓	✓	✗	✗	✓	✗	✓	✓
pgvector+PG (2021) [7]	Ext.	Rel.	✓	✓	✓	✓	✓	✗	✓	✗	✓	✓
SingleStoreDB (2022) [19,105]	Ext.	Rel.	✓	✗	✓	✓	✗	✗	✗	✗	✗	✓
ClickHouse (2023) [20]	Ext.	Rel.	✓	✓	✓	✓	✗	✗	✗	✓	✓	✓
MyScale (2023) [21]	Ext.	Rel.	✓	✓	✓	✓	✗	✗	✓	✗	P	✓

Abbreviations: Ex.=exact k -NN; Ap.=ANN; Rng.=range; Pr.=predicated; Mul.=multi-vector; Bat.=batched; Tab.=table; Tr.=tree; Gr.=graph; Opt.=query optimizer; Nat.=native; Ext.=extended; Vec.=mostly vector; Rel.=relational; U=unknown; P=proprietary.

**Fig. 13** EuclidesDB [6] supports multiple vector spaces so that query results can be more semantically accurate.

embedding models and interact with the system by querying and manipulating non-vector entities.

- *Vald*. Vald [8] is a serverless VDBMS aimed at providing scalable non-predicated vector search. Instead of a single centralized system, Vald spans a Kubernetes²¹ cluster consisting of multiple “agents”. The vector collection is sharded and replicated across the agents to increase throughput and decrease latency via scatter-gather. An individual Vald agent con-

tains an NGT graph index built over its local shard. Vald also supports out-of-place updates on each local replica using a simple update queue.

- *Vearch*. Vearch [17,81] is targeted at image-based search for e-commerce, allowing it to be designed specifically toward this aim. Predicated queries are supported, but they are all executed using post-filtering, considered to be sufficient for the application. Vearch also has no need to support exact queries. Vearch adopts a disaggregated architecture with dedicated search nodes, allowing it to scale read and write capabilities independently.

Pinecone [9] offers a scalable distributed system similar to Vald, but as it is offered as a managed cloud-based service, it can be a more user-friendly option. Chroma [14] is a centralized system similar to EuclidesDB, meant for a single machine.

5.1.2 Mostly Mixed

Mostly-mixed systems aim to support more sophisticated queries compared to mostly-vector systems. In general, they support a greater variety of basic queries, including more support for exact and range queries, in addition to more query variants. These systems likewise have no need for query rewriters or parsers, but some,

²¹ <http://kubernetes.io>

such as Milvus [12,125], Qdrant [10], and Manu [63], make use of query optimization. Other systems, including Weaviate [5], NucliaDB [13], and Marqo [11], also have more sophisticated data and storage models in order to deal with predicated queries and attribute-only queries that go beyond simple retrieval.

- *Milvus and Manu.* Milvus [12,125] is aimed at comprehensive support for vector search queries, and Manu [63] adds additional features on top of Milvus. All three basic query types are supported, in addition to the three query variants. Multiple search indexes are also supported, and predicated queries are handled by a cost-based optimizer.
- *Qdrant.* Qdrant [10] likewise supports a large variety of search queries. For predicated queries, it uses a rule-based optimizer coupled with a custom HNSW index designed for block-first scan.
- *NucliaDB and Marqo.* NucliaDB [13] and Marqo [11] are targeted at document search and retrieval and use vector search to provide semantic retrieval. A key feature is the support for combining keywords and vectors through multi-vector search. Non-vector keyword queries are processed using text specific techniques, resulting in sparse term-frequency vectors. These vectors are then combined with dense feature vectors through an aggregate score in order to conduct multi-vector search.
- *Weaviate.* Weaviate targets document search and retrieval over a graph model. This allows Weaviate to answer non-vector queries, such as retrieving all books written by a certain author, in addition to similarity queries via vector search. Users interact with Weaviate by issuing queries written in GraphQL²².

5.2 Extended

Extended systems inherit all the capabilities of an underlying data management system and are necessarily more complex compared to native systems. But these systems are also more capable. Nearly all the extended systems listed in Table 7 support all three basic query types and multiple indexes, and all of them support query optimization. These systems likewise divide into two subcategories, those where the underlying system is NoSQL and those where it is relational.

5.2.1 NoSQL

Many of the hallmarks of a NoSQL system [50], such as schemaless storage, distributed architecture, and even-

tual consistency, are present in native systems, making a NoSQL extended VDBMS much like a native system.

- *Vespa.* Vespa [18] is a scalable distributed NoSQL system designed for large scale data processing workloads. Unlike native systems, Vespa aims at more general data processing tasks, hence it is equipped with a flexible SQL-like query language. But like native systems, the storage model is simple, provided by an in-house document store. Vespa uses a rule-based query optimizer.
- *Cassandra.* Cassandra [76] is a popular distributed NoSQL system based on a wide column store. Vector search capability will be available in version 5.0²³ by integrating HNSW into the storage layer, implementing scatter-gather across replicas, and extending the Cassandra query language with vector search operators.
- *Databricks.* The Spark-based Databricks²⁴ platform is expected to support vector search, including predicated queries, in an upcoming version.

Aside from Vespa, several other document based NoSQL databases have now been extended to support vector search, including MongoDB²⁵, Cosmos DB²⁶, and Redis²⁷, when coupled with the Redis Stack search extension. Vector search capability has also been extended to NoSQL systems other than document stores. For example, Neo4j²⁸ is a property graph database with experimental vector search capability. So far, only ANN queries are supported, using HNSW.

5.2.2 Relational

For extended relational systems, their features mostly come from the inherent capabilities of relational systems. For example, SQL already is sufficient for expressing (c, k) -search queries, and these queries can already be answered by most relational engines upon adding a user-defined similarity function. This is perhaps exemplified by SingleStore [19,105], which offers exact k -NN and range search through its native relational engine, without any vector search indexes. Subsequently, extended relational systems focus more on tightly integrating vector search capability alongside existing components.

- *SingleStore.* SingleStore [19,105] is a cloud-native hybrid transactional and analytical (HTAP) database

²³ See Cassandra Enhancement Proposal 30 (CEP-30).

²⁴ <http://databricks.com>

²⁵ <http://mongodb.com>

²⁶ <http://cosmos.azure.com>

²⁷ <http://redis.io>

²⁸ <http://neo4j.com>

²² <http://graphql.org>

- [79] based on a distributed row store for fast writes alongside column store replicas for fast reads. Vector search is handled by the native relational engine, extended with functions for calculating dot product and Euclidean distance.
- *PASE*. PASE [139] extends PostgreSQL with a flat quantization index and HNSW index in order to support vector search.
- *pgvector*. The *pgvector* [7] extension for PostgreSQL brings a vector data type, functions over this data type, and flat and HNSW index access methods into PostgreSQL. Vector queries are issued using SQL. If an index is created over a vector column, then queries are answered using the index, yielding approximate results. Otherwise, exact brute force scan is used. Plan selection is performed by the existing PostgreSQL query optimizer using the generic cost estimator, or by calling index-specific cost estimators to refine the estimate, if needed. Other features such as replication, fault tolerance, access controls, concurrency control, and so on, are provided by PostgreSQL.
- *AnalyticDB-V*. AnalyticDB-V [133] adds vector search capability on top of AnalyticDB [141]. This is achieved primarily by introducing vector indexes, VGPQ and HNSW, and by augmenting the cost-based optimizer.
- *ClickHouse and MyScale*. ClickHouse [20] is a columnar database aimed at fast analytics coupled with an asynchronous merge mechanism for fast ingestion. It supports vector queries using ANNOY and HNSW. Likewise, MyScale [21] is a cloud service using ClickHouse as the backend. It adds table-based search indexes including flat indexes and IVFADC, and a proprietary search index called “multi-scale tree graph” (MSTG) which is shown to outperform both IVFADC and HNSW.

5.3 Libraries and Other Systems

Vector search engines and libraries make up a third category in the broader ecosystem. Typically, these are embedded into applications that require vector search, but they lack the capabilities of a full VDBMS.

Search Engines. Apache Lucene [2] is a pluggable search engine that provides sophisticated search capability for embedded applications. Latest versions offer vector search, supported by HNSW. While Lucene itself lacks higher level features such as multi-tenancy, distributed search, and administrative features, many of these are provided by search platforms built on top of Lucene, including Elasticsearch [3], OpenSearch [22], and Solr [23]. These capabilities could make Lucene an

attractive alternative to mostly-vector native VDBMSs²⁹ as it offers similar features and can readily integrate with existing infrastructure.

Libraries. There are also libraries that directly implement specific indexes. For example, KGraph is an implementation of the NN-Descent KNNG. The Microsoft Space Partition Tree and Graph (SPTAG) library [24] combines several techniques, including SPANN and NGT, into one configurable index. Libraries are also available for LSH, including E2LSH³⁰ and FALCONN³¹. Likewise, Meta Faiss [4] offers a selection of indexes, including HNSW, an LSH family for Hamming distance, and quantization-based indexes.

Other Systems. Other systems are aimed at other parts of the broader pipeline. Featureform³² is a middleware aimed at dataset management. Similar to relational ETL tools, it offers a means of organizing the workflows that transform raw data sources into curated datasets to be consumed by downstream applications. Featureform exposes some of these downstream functions through its API. For example, it offers a vector search endpoint that executes a k -NN query over a configured provider, such as Pinecone. On the other hand, Activeloop³³ Deep Lake [65] offers tensor operations directly over a tensor warehouse, making it capable of performing vector search inside the warehouse.

Aside from vector search, vector visualization can also be a useful tool for data analysts. Nomic Atlas³⁴ offers a platform for visualizing high-dimensional vector spaces by projecting them onto two-dimensional plots using a proprietary dimensionality reduction technique.

5.4 Discussion

The designs of these databases cover a spectrum of characteristics involving query processing and vector storage, manifesting in a range of performance and capabilities, as shown in Figure 14.

We offer a few broad remarks. Native mostly-vector systems broadly offer high performance but are targeted at specific workloads, sometimes even specific queries, and thus have relatively limited capability. Meanwhile, native mostly-mixed systems offer more capabilities, notably predicated queries, and some such as Milvus [12, 125], Qdrant [10], and Manu [63] also perform query

²⁹ For a discussion, see <http://arxiv.org/abs/2308.14963>

³⁰ http://www.mit.edu/~andoni/E2LSH_gp1.tar.gz

³¹ <http://github.com/falconn-lib/falconn>

³² <http://featureform.com>

³³ <http://activeloop.ai>

³⁴ <http://nomic.ai>

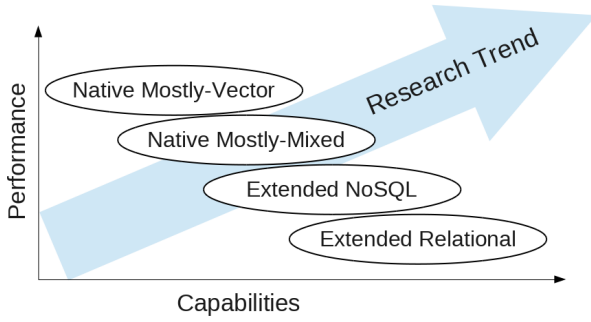


Fig. 14 High-level characteristics of VDBMSs.

optimization. These, along with extended NoSQL systems, achieve a comfortable balance between high performance and search capabilities. On the other hand, extended relational systems offer the most capabilities but possibly less performance. But, as has been mentioned elsewhere³⁵, relational systems are already major components of industrial data infrastructures, and being able to conduct vector search without introducing new systems into the infrastructure is a compelling advantage. The ranking shown in Figure 14 is consistent with empirical observations [37].

Thus, we imagine that future work will target systems that can offer both high performance in addition to offering unified data management capabilities, represented by the arrow in Figure 14.

6 Benchmarks

Comprehensive cross-disciplinary comparisons of vector search algorithms and systems are surprisingly scarce, attributed to the wide range of fields from which search algorithms arise [82]. We point out two notable attempts at benchmarking these algorithms and systems.

In [82], a large number of ANN algorithms are uniformly implemented and evaluated across a range of experimental conditions. These algorithms include LSH, L2H, methods based on quantization, tree-based techniques, and graph-based techniques. The experiments are conducted over 18 datasets, ranging from a few thousand vectors to 10 million vectors, and with dimensions ranging from 100 to 4,096. The feature vectors are derived from real-world image, text, video, and audio collections, as well as synthetically generated. Algorithms are measured on query latency as well as the quality of the result sets based on precision, recall, and two other derivative measures.

In [37], the evaluation is extended to include full VDBMSs in addition to isolated algorithms. Whereas

[82] aims to avoid the effects of different implementations, here these differences are kept in order to more accurately reflect real-world conditions. The standardized evaluation platform and the latest benchmark results are available online³⁶, with results for several VDBMSs. The datasets, however, are smaller in both size and dimensionality than the ones used in [82].

7 Challenges and Open Problems

While much progress has been made on vector data management, some challenges remain unaddressed.

Similarity Score Selection. The semantic quality of different similarity scores is still challenging to understand, and there is no rigorous guidance on how to select which score for which scenarios. Systems like EuclidesDB [6] can be used to experimentally determine the best scores and embedding models, but outside of [130], this problem remains unexplored.

Operator Design. Designing efficient and effective hybrid operators remains challenging. For graph indexes, block-first scan can lead to disconnected components that either need to be repaired or that require new search algorithms for handling this situation. Existing offline blocking techniques [61,12,125,10] are limited to small number of attribute categories. For visit-first scan, estimating the cost of the scan is challenging due to unpredictable backtracking, complicating plan selection.

Incremental Search. Some applications, such as e-commerce and recommender platforms, make use of incremental k -NN search, where k is effectively very large but is retrieved in small increments so that the results appear to be seamlessly delivered to the user. While techniques exist for this type of search [57], it is unclear how to support this search inside vector indexes.

Multi-Vector Search. Multi-vector search is also important for certain applications such as face recognition. Existing techniques tend to use aggregate scores, but this can be inefficient as it multiplies the amount of distance calculations. Generic multi-attribute top- k techniques also are hard to adapt to vector indexes [125], and there are no works on MQMF queries.

Security and Privacy. As vector search becomes increasingly mission-critical, data security and user privacy become more important, especially for VDBMSs that offer managed cloud services. There is thus a need for new techniques that can support private and secure high-dimensional vector search [137].

³⁵ <https://arxiv.org/abs/2308.14963>

³⁶ <http://ann-benchmarks.com>

8 Conclusion

In this paper, we surveyed vector database management systems aimed at fast and accurate vector search, developed in response to recent popularity of dense retrieval for applications such as LLMs and e-commerce. We reviewed considerations for query processing, including similarity scores, query types, and basic operators. We also reviewed the design, search, and maintenance considerations regarding vector search indexes. We described several techniques for query optimization and execution, including plan enumeration, plan selection, operators for predicated or “hybrid” queries, and hardware acceleration. Finally, we discussed several commercial systems and the main benchmarks for supporting experimental comparisons.

References

1. <http://github.com/spotify/annoy>
2. <http://lucene.apache.org>
3. <http://elastic.co>
4. <http://github.com/facebookresearch/faiss>
5. <http://weaviate.io>
6. <http://euclidesdb.readthedocs.io>
7. <http://github.com/pgvector>
8. <http://vald.vdaas.org>
9. <http://pinecone.io>
10. <http://qdrant.tech>
11. <http://marqo.ai>
12. <http://milvus.io>
13. <http://nuclia.com>
14. <http://trychroma.com>
15. <http://github.com/flann-lib/flann>
16. <http://github.com/yahoojapan/NGT>
17. <http://github.com/vearch>
18. <http://vespa.ai>
19. <http://singlestore.com>
20. <http://clickhouse.com>
21. <http://myscale.com>
22. <http://opensearch.org>
23. <http://solr.apache.org>
24. <http://github.com/microsoft/SPTAG>
25. Abdelkader, A., Arya, S., da Fonseca, G.D., Mount, D.M.: Approximate nearest neighbor searching with non-Euclidean and weighted distances. In: SODA, pp. 355–372 (2019)
26. Adelson-Velsky, G., Landis, E.: An algorithm for the organization of information. *Proc. USSR Acad. Sci.* **146**, 263–266 (1962)
27. Aggarwal, C.C., Hinneburg, A., Keim, D.A.: On the surprising behavior of distance metrics in high dimensional space. In: ICDT (2001)
28. Agrell, E., Allen, B.: On the best lattice quantizers. *IEEE Trans. Inform. Theory* (2023)
29. Andoni, A., Indyk, P.: Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. In: FOCS, pp. 459–468 (2006)
30. Andoni, A., Indyk, P.: Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Commun. ACM* **51**(1), 117–122 (2008)
31. Andoni, A., Indyk, P., Laarhoven, T., Razenshteyn, I., Schmidt, L.: Practical and optimal LSH for angular distance. In: NeurIPS, pp. 1225–1233 (2015)
32. Andoni, A., Indyk, P., Razenshteyn, I.: Approximate nearest neighbor search in high dimensions. In: ICM, pp. 3287–3318 (2018)
33. Andoni, A., Razenshteyn, I.: Optimal data-dependent hashing for approximate near neighbors. In: STOC, pp. 793–801 (2015)
34. André, F., Kermarrec, A.M., Le Scouarnec, N.: Accelerated nearest neighbor search with Quick ADC. In: ICMR (2017)
35. André, F., Kermarrec, A.M., Le Scouarnec, N.: Quicker ADC: Unlocking the hidden potential of product quantization with SIMD. *IEEE Trans. Pattern Anal. and Mach. Intell.* **43**(5), 1666–1677 (2021)
36. Asai, A., Min, S., Zhong, Z., Chen, D.: Retrieval-based language models and applications. In: ACL (2023)
37. Aumüller, M., Bernhardtsson, E., Faithfull, A.: ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inform. Syst.* **87** (2020)
38. Azizi, I., Echihiabi, K., Palpanas, T.: ELPIS: Graph-based similarity search for scalable data science. *Proc. VLDB Endow.* **16**(6), 1548–1559 (2023)
39. Bentley, J.L.: Multidimensional binary search trees used for associative searching. *Commun. ACM* **18**(9), 509–517 (1975)
40. Berg, M., Cheong, O., Kreveld, M., Overmars, M.: *Computational Geometry: Algorithms and Applications*, 3 edn. Springer-Verlag (2008)
41. Beyer, K., Goldstein, J., Ramakrishnan, R., Shaft, U.: When is “nearest neighbor” meaningful? In: ICDT (1999)
42. Chang, W.C., Yu, F.X., Chang, Y.W., Yang, Y., Kumar, S.: Pre-training tasks for embedding-based large-scale retrieval. In: ICLR (2020)
43. Chen, L., Gao, Y., Song, X., Li, Z., Zhu, Y., Miao, X., Jensen, C.S.: Indexing metric spaces for exact similarity search. *ACM Comput. Surv.* **55**(6) (2022)
44. Chen, Q., Zhao, B., Wang, H., Li, M., Liu, C., Li, Z., Yang, M., Wang, J., Yang, M., Wang, J.: SPANN: Highly-efficient billion-scale approximate nearest neighbor search. In: NeurIPS (2021)
45. Ciaccia, P., Patella, M., Zezula, P.: M-Tree: An efficient access method for similarity search in metric spaces. *Proc. VLDB Endow.* pp. 426–435 (1997)
46. Cormode, G., Garofalakis, M., Haas, P.J., Jermaine, C.: *Synopses for Massive Data: Samples, Histograms, Wavelets, Sketches*. Now Publishers (2011)
47. Dasgupta, S., Freund, Y.: Random projection trees and low dimensional manifolds. In: STOC, pp. 537–546 (2008)
48. Dasgupta, S., Sinha, K.: Randomized partition trees for exact nearest neighbor search. In: COLT, pp. 317–337 (2013)
49. Datar, M., Immorlica, N., Indyk, P., Mirrokni, V.S.: Locality-sensitive hashing scheme based on p-stable distributions. In: SCG, pp. 253–262 (2004)
50. Davoudian, A., Chen, L., Liu, M.: A survey on NoSQL stores. *ACM Comput. Surv.* **51**(2) (2018)
51. Dearholt, D., Gonzales, N., Kurup, G.: Monotonic search networks for computer vision databases. In: ACSSC, pp. 548–553 (1988)
52. Dong, W., Charikar, M., Li, K.: Efficient k -nearest neighbor graph construction for generic similarity measures. In: WWW (2011)
53. Echihiabi, K., Zoumpatianos, K., Palpanas, T.: New trends in high-D vector similarity search: AI-driven, progressive, and distributed. *Proc. VLDB Endow.* **14**(12), 3198–3201 (2021)

54. Echihiabi, K., Zoumpatianos, K., Palpanas, T., Benbrahim, H.: Return of the Lernaean Hydra: Experimental evaluation of data series approximate similarity search. *Proc. VLDB Endow.* **13**(3), 403–420 (2019)
55. Edelsbrunner, H., Shah, N.R.: Incremental topological flipping works for regular triangulations. *Algorithmica* **15**, 223–241 (1996)
56. Eppstein, D., Paterson, M.S., Yao, F.F.: On nearest-neighbor graphs. *Discrete Comput. Geom.* **17**, 263–282 (1997)
57. Fisher, D., Popov, I., Drucker, S., Schraefel, M.C.: Trust me, I'm partially right: Incremental visualization lets analysts explore large datasets faster. In: *SIGCHI*, pp. 1673–1682 (2012)
58. Fu, C., Xiang, C., Wang, C., Cai, D.: Fast approximate nearest neighbor search with the navigating spreading-out graph. *Proc. VLDB Endow.* **12**(5), 461–474 (2019)
59. Ge, T., He, K., Ke, Q., Sun, J.: Optimized product quantization for approximate nearest neighbor search. In: *CVPR*, pp. 2946–2953 (2013)
60. Gersho, A., Gray, R.M.: *Vector Quantization and Signal Compression*. Springer (1991)
61. Gollapudi, S., Karia, N., Sivashankar, V., Krishnaswamy, R., Begwani, N., Raz, S., Lin, Y., Zhang, Y., Mahapatro, N., Srinivasan, P., Singh, A., Simhadri, H.V.: Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In: *WWW* (2023)
62. Gray, R.M.: Vector quantization. *IEEE ASSP Mag.* **1**(2), 4–29 (1984)
63. Guo, R., Luan, X., Xiang, L., Yan, X., Yi, X., Luo, J., Cheng, Q., Xu, W., Luo, J., Liu, F., Cao, Z., Qiao, Y., Wang, T., Tang, B., Xie, C.: Manu: A cloud native vector database management system. *Proc. VLDB Endow.* **15**(12), 3548–3561 (2022)
64. Guo, R., Sun, P., Lindgren, E., Geng, Q., Simcha, D., Chern, F., Kumar, S.: Accelerating large-scale inference with anisotropic vector quantization. In: *ICML* (2020)
65. Hambardzumyan, S., Tuli, A., Ghukasyan, L., Rahman, F., Topchyan, H., Isayan, D., McQuade, M., Harutyunyan, M., Hakobyan, T., Stranic, I., Buniatyan, D.: Deep Lake: A lakehouse for deep learning. In: *CIDR* (2023)
66. Harwood, B., Drummond, T.: FANNG: Fast approximate nearest neighbour graphs. In: *CVPR* (2016)
67. Indyk, P., Motwani, R.: Approximate nearest neighbors: Towards removing the curse of dimensionality. In: *STOC*, pp. 604–613 (1998)
68. Jaromczyk, J., Toussaint, G.: Relative neighborhood graphs and their relatives. *Proc. IEEE* **80**(9), 1502–1517 (1992)
69. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. and Mach. Intell.* **33**(1), 117–128 (2011)
70. Johnson, J., Douze, M., Jégou, H.: Billion-scale similarity search with GPUs. *IEEE Trans. Big Data* **7**(3), 535–547 (2021)
71. Jurafsky, D., Martin, J.H.: *Speech and Language Processing*, 2nd edn. Prentice-Hall (2009)
72. Keivani, O., Sinha, K., Ram, P.: Improved maximum inner product search with better theoretical guarantee using randomized partition trees. *Mach. Learn.* **107**, 1069–1094 (2018)
73. Kim, Y.: Applications and future of dense retrieval in industry. In: *SIGIR*, pp. 3373–3374 (2022)
74. Kleinberg, J.M.: Navigation in a small world. *Nature* **406**, 845 (2000)
75. Laarhoven, T.: Graph-based time-space trade-offs for approximate near neighbors. In: *SoCG* (2018)
76. Lakshman, A., Malik, P.: Cassandra: A decentralized structured storage system. *SIGOPS Oper. Syst. Rev.* **44**(2), 35–40 (2010)
77. Lee, D., Wong, C.: Worst-case analysis for region and partial region searches in multidimensional binary search trees and balanced quad trees. *Acta Inform.* **9**, 23–29 (1977)
78. Leskovec, J., Rajaraman, A., Ullman, J.: *Mining of Massive Datasets*, 3rd edn. Cambridge University Press (2014)
79. Li, G., Zhang, C.: HTAP databases: What is new and what is next. In: *SIGMOD*, pp. 2483–2488 (2022)
80. Li, H., Ai, Q., Zhan, J., Mao, J., Liu, Y., Liu, Z., Cao, Z.: Constructing tree-based index for efficient and effective dense retrieval. In: *SIGIR* (2023)
81. Li, J., Liu, H., Gui, C., Chen, J., Ni, Z., Wang, N., Chen, Y.: The design and implementation of a real time visual search system on JD e-commerce platform. In: *Middleware*, pp. 9–16 (2018)
82. Li, W., Zhang, Y., Sun, Y., Wang, W., Li, M., Zhang, W., Lin, X.: Approximate nearest neighbor search on high dimensional data — Experiments, analyses, and improvement. *IEEE Trans. Knowl. Data Eng.* **32**(8), 1475–1488 (2020)
83. Lindholm, E., Nickolls, J., Oberman, S., Montrym, J.: NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro.* **28**(2), 39–55 (2008)
84. Lipton, R.J., Tarjan, R.E.: Applications of a planar separator theorem. *SIAM J. Comput.* **9**(3), 615–627 (1980)
85. Liu, T., Moore, A.W., Gray, A., Yang, K.: An investigation of practical approximate nearest neighbor algorithms. In: *NeurIPS*, pp. 825–832 (2004)
86. Lloyd, S.: Least squares quantization in PCM. *IEEE Trans. Inform. Theory* **28**(2), 129–137 (1982)
87. Luo, C., Carey, M.J.: LSM-Based storage techniques: A survey. *VLDB J.* **29**(1), 393–418 (2019)
88. Lv, Q., Josephson, W., Wang, Z., Charikar, M., Li, K.: Multi-probe LSH: Efficient indexing for high-dimensional similarity search. *Proc. VLDB Endow.* pp. 950–961 (2007)
89. Malkov, Y., Ponomarenko, A., Logvinov, A., Krylov, V.: Approximate nearest neighbor algorithm based on navigable small world graphs. *Inform. Syst.* **45**, 61–68 (2014)
90. Malkov, Y., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. and Mach. Intell.* **42**(4), 824–836 (2020)
91. Matsui, Y., Uchida, Y., Jégou, H., Satoh, S.: A survey of product quantization. *ITE Trans. Media Technol. and Appl.* **6**(1), 2–10 (2018)
92. Meiser, S.: Point location in arrangements of hyperplanes. *Inform. Comput.* **106**(2), 286–303 (1993)
93. Meng, J., Wang, H., Xu, J., Ogihara, M.: ONe Index for All Kernels (ONIAK): A zero re-indexing LSH solution to ANNS-ALT (After Linear Transformation). *Proc. VLDB Endow.* **15**(13), 3937–3949 (2022)
94. Mirkes, E.M., Allohbi, J., Gorban, A.: Fractional norms and quasinorms do not help to overcome the curse of dimensionality. *Entropy* **22**(10) (2020)
95. Mitra, B., Craswell, N.: An introduction to neural information retrieval. *Found. Trends Inf. Retr.* **13**(1), 1–126 (2018)
96. Muja, M., Lowe, D.G.: FLANN: Fast library for approximate nearest neighbors. In: *VISAPP* (2009)
97. Navarro, G.: Searching in metric spaces by spatial approximation. *VLDB J.* (2002)
98. Norouzi, M., Fleet, D.J.: Cartesian k -means. In: *CVPR* (2013)
99. O'Neil, P., Cheng, E., Gawlick, D., O'Neil, E.: The log-structured merge-tree (LSM-tree). *Acta Inform.* **33**, 351–385 (1996)

100. O'Donnell, R., Wu, Y., Zhou, Y.: Optimal lower bounds for locality-sensitive hashing (except when q is tiny). *ACM Trans. Comput. Theory* **6**(1), 1–13 (2014)
101. Paredes, R., Chávez, E.: Using the k -nearest neighbor graph for proximity searching in metric spaces. In: *SPIRE*, pp. 127–138 (2005)
102. Paredes, R., Chávez, E., Figueroa, K., Navarro, G.: Practical construction of k -nearest neighbor graphs in metric spaces. In: *WEA* (2006)
103. Pouyanfar, S., Sadiq, S., Yan, Y., Tian, H., Tao, Y., Reyes, M.P., Shyu, M.L., Chen, S.C., Iyengar, S.S.: A survey on deep learning: Algorithms, techniques, and applications. *ACM Comput. Surv.* **51**(5) (2018)
104. Prokhorenkova, L., Shekhovtsov, A.: Graph-based nearest neighbor search: From practice to theory. In: *ICML* (2020)
105. Prout, A., Wang, S.P., Victor, J., Sun, Z., Li, Y., Chen, J., Bergeron, E., Hanson, E., Walzer, R., Gomes, R., Shangunov, N.: Cloud-native transactions and analytics in SingleStore. In: *SIGMOD*, pp. 2340–2352 (2022)
106. Qin, J., Wang, W., Xiao, C., Zhang, Y.: Similarity query processing for high-dimensional data. *Proc. VLDB Endow.* **13**(12), 3437–3440 (2020)
107. Qin, J., Wang, W., Xiao, C., Zhang, Y., Wang, Y.: High-dimensional similarity query processing for data science. In: *KDD*, pp. 4062–4063 (2021)
108. Ram, P., Sinha, K.: Revisiting kd -tree for nearest neighbor search. In: *KDD*, pp. 1378–1388 (2019)
109. Rubinstein, A.: Hardness of approximate nearest neighbor search. In: *STOC*, pp. 1260–1268 (2018)
110. Salakhutdinov, R.R., Hinton, G.E.: Learning a nonlinear embedding by preserving class neighbourhood structure. In: *AISTATS* (2007)
111. Sellis, T., Roussopoulos, N., Faloutsos, C.: Multidimensional access methods: Trees have grown everywhere. *Proc. VLDB Endow.* pp. 13–14 (1997)
112. Silpa-Anan, C., Hartley, R.: Optimised KD-trees for fast image descriptor matching. In: *CVPR* (2008)
113. Sivic, Zisserman: Video Google: A text retrieval approach to object matching in videos. In: *ICCV*, pp. 1470–1477 (2003)
114. Su, T.H., Chang, R.C.: On constructing the relative neighborhood graphs in Euclidean k -dimensional spaces. *Computing* **46**, 121–130 (1991)
115. Subramanya, S.J., Devvrit, Kadekodi, R., Krishnaswamy, R., Simhadri, H.: DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In: *NeurIPS* (2019)
116. Tagliabue, J., Greco, C.: (Vector) Space is not the final frontier: Product search as program synthesis. In: *SIGIR* (2023)
117. Teflioudi, C., Gemulla, R.: Exact and approximate maximum inner product search with LEMP. *ACM Trans. Database Syst.* **42**(1) (2016)
118. Toussaint, G.T.: The relative neighbourhood graph of a finite planar set. *Pattern Recognit.* **12**(4), 261–268 (1980)
119. Vaidya, P.M.: An $O(n \log n)$ algorithm for the all-nearest-neighbors problem. *Discrete Comput. Geom.* **4**, 101–115 (1989)
120. Vempala, S.S.: Randomly-oriented k - d trees adapt to intrinsic dimension. In: *LIPICs* (2012)
121. Wang, F., Sun, J.: Survey on distance metric learning and dimensionality reduction in data mining. *Data Min. Knowl. Disc.* **29**, 534–564 (2015)
122. Wang, J., Li, S.: Query-driven iterated neighborhood graph search for large scale indexing. In: *MM*, pp. 179–188 (2012)
123. Wang, J., Wang, J., Zeng, G., Tu, Z., Gan, R., Li, S.: Scalable k -NN graph construction for visual descriptors. In: *CVPR*, pp. 1106–1113 (2012)
124. Wang, J., Wang, N., Jia, Y., Li, J., Zeng, G., Zha, H., Hua, X.S.: Trinary-projection trees for approximate nearest neighbor search. *IEEE Trans. Pattern Anal. and Mach. Intell.* **36**(2), 388–403 (2014)
125. Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., Yu, K., Yuan, Y., Zou, Y., Long, J., Cai, Y., Li, Z., Zhang, Z., Mo, Y., Gu, J., Jiang, R., Wei, Y., Xie, C.: Milvus: A purpose-built vector data management system. In: *SIGMOD*, pp. 2614–2627 (2021)
126. Wang, J., Zhang, Q.: Disaggregated database systems. In: *SIGMOD*, pp. 37–44 (2023)
127. Wang, J., Zhang, T., Song, J., Sebe, N., Shen, H.T.: A survey on learning to hash. *IEEE Trans. Pattern Anal. and Mach. Intell.* **40**(4), 769–790 (2018)
128. Wang, M., Xu, X., Yue, Q., Wang, Y.: A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* **14**(11), 1964–1978 (2021)
129. Wang, R., Deng, D.: DeltaPQ: Lossless product quantization code compression for high dimensional similarity search. *Proc. VLDB Endow.* **13**(13), 3603–3616 (2020)
130. Wang, W., Guo, Y., Shen, C., Ding, S., Liao, G., Fu, H., Prabhakar, P.K.: Integrity and junkiness failure handling for embedding-based retrieval: A case study in social network search. In: *SIGIR* (2023)
131. Watts, D.J., Strogatz, S.H.: Collective dynamics of ‘small-world’ networks. *Nature* **393**, 440–442 (1998)
132. Weber, R., Schek, H.J., Blott, S.: A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces. *Proc. VLDB Endow.* pp. 194–205 (1998)
133. Wei, C., Wu, B., Wang, S., Lou, R., Zhan, C., Li, F., Cai, Y.: AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data. *Proc. VLDB Endow.* **13**(12), 3152–3165 (2020)
134. Weiss, Y., Torralba, A., Fergus, R.: Spectral hashing. In: *NeurIPS*, pp. 1753–1760 (2008)
135. Williams, R.: On the difference between closest, furthest, and orthogonal pairs: Nearly-linear vs barely-subquadratic complexity. In: *SODA*, pp. 1207–1215 (2018)
136. Wu, W., He, J., Qiao, Y., Fu, G., Liu, L., Yu, J.: HQANN: Efficient and robust similarity search for hybrid queries with structured and unstructured constraints. In: *CIKM* (2022)
137. Xue, W., Li, H., Peng, Y., Cui, J., Shi, Y.: Secure k nearest neighbors query for high-dimensional vectors in outsourced environments. *IEEE Trans. Big Data* **4**(4), 586–599 (2018)
138. Yandex, A.B., Lempitsky, V.: Efficient indexing of billion-scale datasets of deep descriptors. In: *CVPR*, pp. 2055–2063 (2016)
139. Yang, W., Li, T., Fang, G., Wei, H.: PASE: PostgreSQL ultra-high-dimensional approximate nearest neighbor search extension. In: *SIGMOD*, pp. 2241–2253 (2020)
140. Yianilos, P.N.: Data structures and algorithms for nearest neighbor search in general metric spaces. In: *SODA*, pp. 311–321 (1993)
141. Zhan, C., Su, M., Wei, C., Peng, X., Lin, L., Wang, S., Chen, Z., Li, F., Pan, Y., Zheng, F., Chai, C.: AnalyticDB: Real-time OLAP database system at Alibaba Cloud. *Proc. VLDB Endow.* **12**(12), 2059–2070 (2019)
142. Zhang, H., Cao, L., Yan, Y., Madden, S., Rundensteiner, E.A.: Continuously adaptive similarity search. In: *SIGMOD*, pp. 2601–2616 (2020)
143. Zhang, W., Ji, J., Zhu, J., Li, J., Xu, H., Zhang, B.: BitHash: An efficient bitwise locality sensitive hashing method with applications. *Knowl. Based Syst.* (2016)
144. Zhu, Y., Chen, L., Gao, Y., Jensen, C.S.: Pivot selection algorithms in metric spaces: A survey and experimental study. *VLDB J.* **31**(1) (2022)